🚚 Delhivery Logistics — Business Case Study

EDA, Feature Engineering & Business Intelligence Report

Dataset: Delhivery Supply-Chain Data | **Period:** Sep–Oct 2018 | **Rows:** 1,44,867 | **Columns:** 24

⚙️ Setup — Install & Import Libraries

```
# Install any missing libraries (Colab-safe)
# !pip install -q pandas numpy matplotlib seaborn scipy scikit-learn

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from scipy import stats
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScale
import warnings

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 30)
pd.set_option('display.float_format', '{:.3f}'.format)

# ── Consistent plot style ──────────────────────────────────────────────
sns.set_theme(style='whitegrid', palette='muted', font_scale=1.1)
PALETTE = ['#4C72B0','#DD8452','#55A868','#C44E52','#8172B2','#937860']
plt.rcParams.update({'figure.dpi': 120, 'axes.titlesize': 13,
                     'axes.labelsize': 11, 'figure.figsize': (12, 5)})

print("✅ All libraries imported successfully.")
print(f"   pandas {pd.__version__} | numpy {np.__version__}")
```

```
✅ All libraries imported successfully.
   pandas 2.2.2 | numpy 2.0.2
```

📂 Load Dataset

```
from google.colab import files
uploaded = files.upload()

# ── Read CSV ──────────────────────────────────────────────────────────
df = pd.read_csv('delhivery_data.csv')          # change path if needed
df_original = df.copy()                          # keep a pristine backup

print(f"Dataset loaded: {df.shape[0]:,} rows × {df.shape[1]} columns")
df.head(3)
```

Choose Files delhivery_data.csv

delhivery_data.csv(text/csv) - 47723907 bytes, last modified: 5/26/2026 - 100% done

```
Saving delhivery_data.csv to delhivery_data.csv
Dataset loaded: 144,867 rows × 24 columns
```

	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_
0	training	2018-09-20 2:35:36	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNaga (Gu
1	training	2018-09-20 2:35:36	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNaga (Gu
2	training	2018-09-20 2:35:36	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	trip-153741093647649320	IND388121AAA	Anand_VUNaga (Gu

---

1. Problem Statement & Exploratory Data Analysis

1.1 Problem Statement

Delhivery is India's largest and fastest-growing logistics company, operating a supply-chain network of hubs, distribution points, and delivery centers across the country.

Business Problem: The company wants to understand the **efficiency of its delivery network** by:

1. Comparing **actual delivery time/distance** against **OSRM (OpenStreetMap Routing Machine) estimates**.
2. Identifying **corridors, routes, and hubs** that consistently underperform.
3. Building a **clean, feature-rich dataset** that can later feed a machine-learning model predicting delivery times more accurately.

Key Questions:

- Where do the biggest delays occur (which routes, which states)?
- How much does actual time deviate from map-estimated time, and why?
- Which corridors are busiest and which are least efficient?
- What features best predict actual delivery time?

Additional Context:

- Each row represents **one route segment** of a multi-segment trip.
- Multiple rows share the same `trip_uuid` — aggregating them reconstructs the full trip.
- `factor = actual_time / osrm_time` is the key efficiency metric: **>1 means delays**, <1 means faster than expected.

1.2 Data Overview — Shape, Types, Missing Values & Statistical Summary

```
print("=" * 60)
print(f"Shape: {df.shape[0]:,} rows  ×  {df.shape[1]} columns")
print("=" * 60)
print()

# ── Data types ──────────────────────────────────────────────
print("── Column Data Types ──────────────────────────────────────")
print(df.dtypes.to_string())
```

```
============================================================
Shape: 144,867 rows  ×  24 columns
============================================================

── Column Data Types ──────────────────────────────────────
data                            object
trip_creation_time              object
route_schedule_uuid             object
route_type                      object
trip_uuid                       object
source_center                   object
source_name                     object
destination_center              object
```

```
destination_name                object
od_start_time                   object
od_end_time                     object
start_scan_to_end_scan           int64
is_cutoff                         bool
cutoff_factor                    int64
cutoff_timestamp                object
actual_distance_to_destination  float64
actual_time                      int64
osrm_time                        int64
osrm_distance                   float64
factor                          float64
segment_actual_time              int64
segment_osrm_time                int64
segment_osrm_distance           float64
segment_factor                  float64
```

```
# ── Convert appropriate columns to 'category' dtype ──────────────────────
cat_cols = ['data', 'route_type', 'is_cutoff',
            'source_center', 'source_name',
            'destination_center', 'destination_name']

for col in cat_cols:
    df[col] = df[col].astype('category')

# ── Convert datetime columns ─────────────────────────────────────────────
dt_cols = ['trip_creation_time', 'od_start_time', 'od_end_time', 'cutoff_t
for col in dt_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce')

print("✅ Categorical & datetime conversions done.")
print()
print("Updated dtypes:")
print(df.dtypes.to_string())
```

```
✅ Categorical & datetime conversions done.

Updated dtypes:
data                                category
trip_creation_time                datetime64[ns]
route_schedule_uuid                       object
route_type                          category
trip_uuid                                 object
source_center                       category
source_name                         category
destination_center                  category
destination_name                    category
od_start_time                     datetime64[ns]
od_end_time                       datetime64[ns]
start_scan_to_end_scan                     int64
is_cutoff                           category
cutoff_factor                              int64
cutoff_timestamp                  datetime64[ns]
actual_distance_to_destination           float64
actual_time                                int64
osrm_time                                  int64
osrm_distance                            float64
factor                                   float64
segment_actual_time                        int64
segment_osrm_time                          int64
segment_osrm_distance                    float64
segment_factor                           float64
```

```
# ── Missing value analysis ──────────────────────────────────────────────
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
miss_df = pd.DataFrame({'Missing Count': missing, 'Missing %': missing_pct}
miss_df = miss_df[miss_df['Missing Count'] > 0]

print("── Missing Values ──────────────────────────────────────────────────")
print(miss_df.to_string())
print()
```

```
print("✅ Only 'source_name' and 'destination_name' have minor missing valu
print("   These are name-labels only; the center-code columns have no nulls
```

```
── Missing Values ──────────────────────────────────────
                 Missing Count  Missing %
source_name                293      0.200
destination_name           261      0.180

✅ Only 'source_name' and 'destination_name' have minor missing values (~0.2%).
   These are name-labels only; the center-code columns have no nulls.
```

```
# ── Statistical Summary ──────────────────────────────────────────────────
print("── Statistical Summary of Numerical Columns ──────────────────────")
num_summary = df.select_dtypes(include='number').describe().T
num_summary['range'] = num_summary['max'] - num_summary['min']
num_summary['IQR']   = num_summary['75%'] - num_summary['25%']
print(num_summary.round(3).to_string())
```

```
── Statistical Summary of Numerical Columns ──────────────────
                                  count     mean      std      min     25%     50%      75%      max     ra
start_scan_to_end_scan         144867.000 961.263 1037.013   20.000 161.000 449.000 1634.000 7898.000 7878.
cutoff_factor                  144867.000 232.927  344.756    9.000  22.000  66.000  286.000 1927.000 1918.
actual_distance_to_destination 144867.000 234.073  344.990    9.000  23.356  66.127  286.709 1927.448 1918.
actual_time                    144867.000 416.928  598.104    9.000  51.000 132.000  513.000 4532.000 4523.
osrm_time                      144867.000 213.868  308.011    6.000  27.000  64.000  257.000 1686.000 1680.
osrm_distance                  144867.000 284.771  421.119    9.008  29.915  78.526  343.193 2326.199 2317.
factor                         144867.000   2.120    1.715    0.144   1.604   1.857    2.213   77.387   77.
segment_actual_time            144867.000  36.196   53.571 -244.000  20.000  29.000   40.000 3051.000 3295.
segment_osrm_time              144867.000  18.508   14.776    0.000  11.000  17.000   22.000 1611.000 1611.
segment_osrm_distance          144867.000  22.829   17.861    0.000  12.070  23.513   27.813 2191.404 2191.
segment_factor                 144867.000   2.218    4.848  -23.444   1.348   1.684    2.250  574.250  597.
```

```
# ── Categorical column summaries ─────────────────────────────────────────
for col in ['data', 'route_type', 'is_cutoff']:
    print(f"\n── {col} ──────────────────────────────────────")
    vc = df[col].value_counts()
    pct = (vc / len(df) * 100).round(2)
    print(pd.concat([vc, pct.rename('%')], axis=1).to_string())

print()
print(f"Unique source centers      : {df['source_center'].nunique():,}")
print(f"Unique destination centers : {df['destination_center'].nunique():,}
print(f"Unique trips               : {df['trip_uuid'].nunique():,}")
print(f"Unique scheduled routes    : {df['route_schedule_uuid'].nunique():,
print(f"Date range : {df['trip_creation_time'].min().date()}  →  {df['trip_
```

```
── data ──────────────────────────────
          count      %
data
training  104858 72.380
test       40009 27.620

── route_type ──────────────────────────────
            count      %
route_type
FTL         99660 68.790
Carting     45207 31.210

── is_cutoff ──────────────────────────────
            count      %
is_cutoff
True       118749 81.970
False       26118 18.030

Unique source centers      : 1,508
Unique destination centers : 1,481
Unique trips               : 14,817
Unique scheduled routes    : 1,504
Date range : 2018-09-12  →  2018-10-03
```

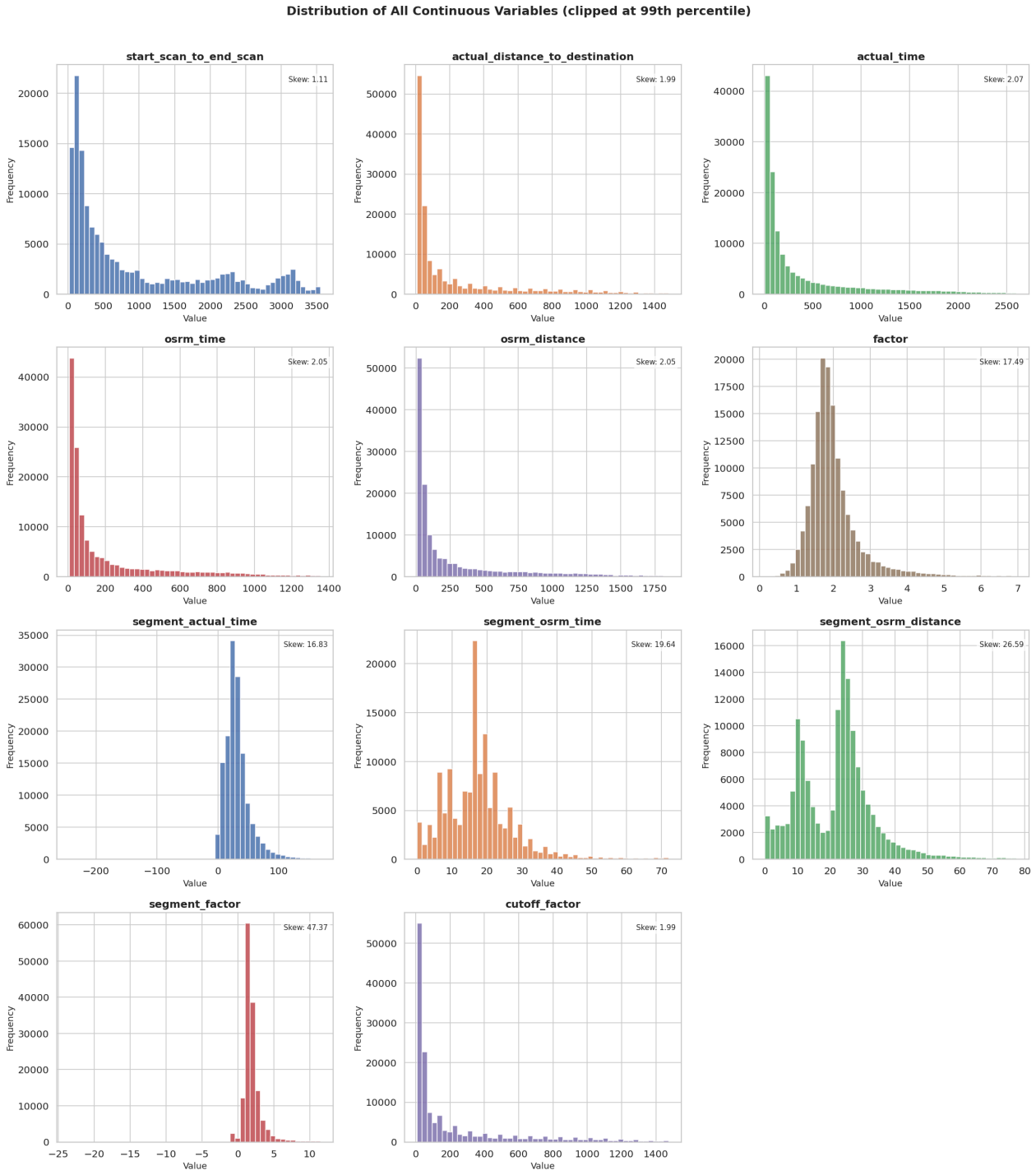
📝 **Observations — 1.2:**

- Dataset has **1,44,867 rows** representing individual route **segments** (not full trips).
- There are **14,817 unique trips** — each trip has on average ~9.8 segments.
- `factor` (= actual_time / osrm_time) is the core efficiency KPI; mean ≈ 2.12, meaning deliveries take **~2× longer** than map estimates on average.
- `source_name` and `destination_name` have **<0.3% missing** values — negligible, safe to impute or drop.
- `segment_factor` has **negative values** (2,365 rows) — this is a data anomaly worth investigating.
- `start_scan_to_end_scan` ranges from 20 to 7,898 minutes (~5.5 days) — potential outliers exist.
- The dataset covers a **22-day window** (Sep 12 – Oct 3, 2018).

1.3 Visual Analysis

Distribution Plots — Continuous Variables

```
# ── Distribution plots of all continuous variables ──────────────────────
num_cols = ['start_scan_to_end_scan', 'actual_distance_to_destination',
            'actual_time', 'osrm_time', 'osrm_distance',
            'factor', 'segment_actual_time', 'segment_osrm_time',
            'segment_osrm_distance', 'segment_factor', 'cutoff_factor']

fig, axes = plt.subplots(4, 3, figsize=(18, 20))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    ax = axes[i]
    data = df[col].dropna()
    # clip extreme outliers for plotting clarity
    q99 = data.quantile(0.99)
    clipped = data[data <= q99]
    ax.hist(clipped, bins=50, color=PALETTE[i % len(PALETTE)], edgecolor='v
    ax.set_title(f'{col}', fontweight='bold')
    ax.set_xlabel('Value')
    ax.set_ylabel('Frequency')
    skew_val = data.skew()
    ax.text(0.98, 0.95, f'Skew: {skew_val:.2f}', transform=ax.transAxes,
            ha='right', va='top', fontsize=9,
            bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.

# Hide the last unused subplot
axes[-1].set_visible(False)
plt.suptitle('Distribution of All Continuous Variables (clipped at 99th per
             fontsize=15, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('01_distributions.png', bbox_inches='tight', dpi=120)
plt.show()
print("\n📊 Plot saved: 01_distributions.png")
```



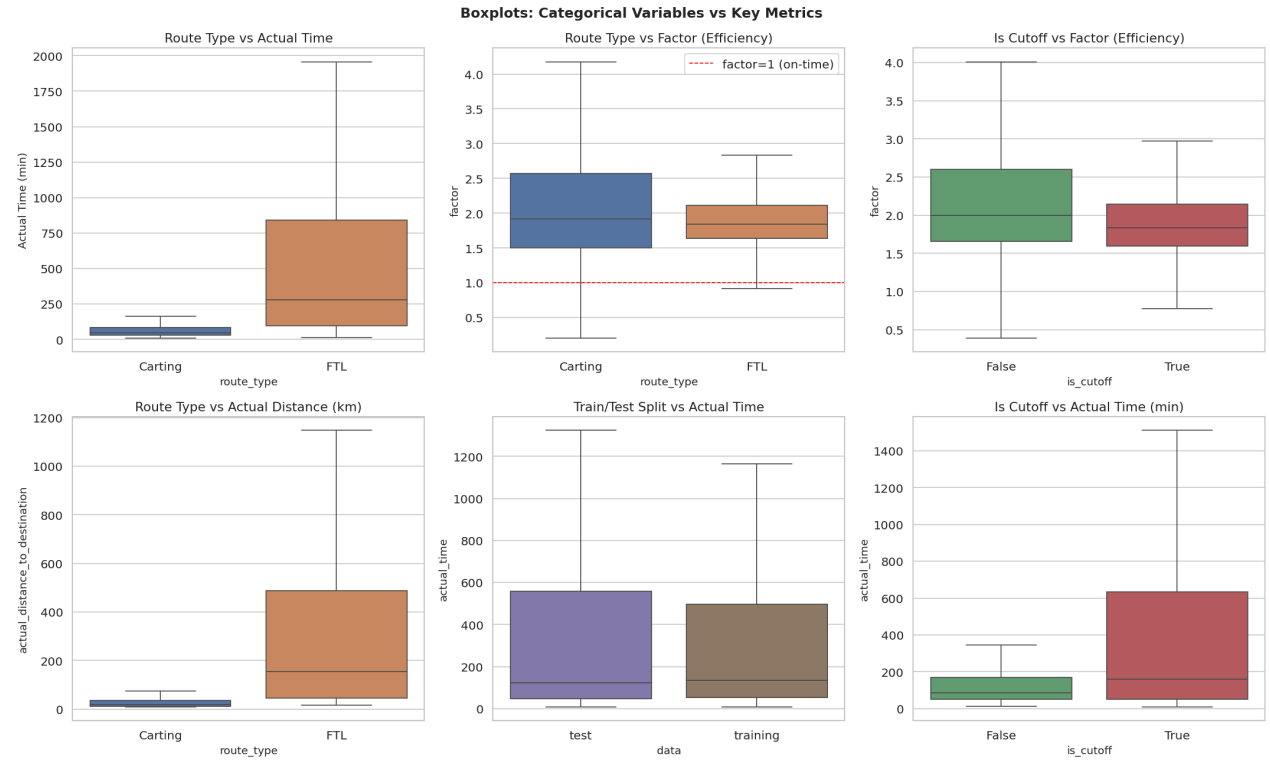
Plot saved: 01_distributions.png

Observations — Distribution Plots:

- `actual_time`, `osrm_time`, `actual_distance_to_destination`, `osrm_distance` — all heavily **right-skewed**, indicating most segments are short but a few are very long (long-haul routes).
- `factor` and `segment_factor` — right-skewed with a peak near 1.5–2.0, confirming systemic delays vs. OSRM estimates.
- `start_scan_to_end_scan` — bimodal distribution; shorter scans (~~100 min) represent carting segments, longer~~ (1500+ min) represent FTL long-haul trips.
- `cutoff_factor` — unusual uniform-like spread due to it representing minutes-to-cutoff from 9 to 1927 minutes, reflecting diverse delivery windows.
- `segment_factor` — has a long right tail with some extreme values (>100), indicating specific segments with very unusual delays.

Boxplots — Categorical vs. Numerical Variables

```
# ── Boxplots: Categorical variables vs key numeric metrics ──────────────
fig, axes = plt.subplots(2, 3, figsize=(18, 11))

# 1. route_type vs actual_time
sns.boxplot(data=df, x='route_type', y='actual_time',
            showfliers=False, palette=['#4C72B0','#DD8452'], ax=axes[0,0])
axes[0,0].set_title('Route Type vs Actual Time')
axes[0,0].set_ylabel('Actual Time (min)')

# 2. route_type vs factor
sns.boxplot(data=df, x='route_type', y='factor',
            showfliers=False, palette=['#4C72B0','#DD8452'], ax=axes[0,1])
axes[0,1].set_title('Route Type vs Factor (Efficiency)')
axes[0,1].axhline(1.0, color='red', linestyle='--', linewidth=1, label='fac
axes[0,1].legend()

# 3. is_cutoff vs factor
sns.boxplot(data=df, x='is_cutoff', y='factor',
            showfliers=False, palette=['#55A868','#C44E52'], ax=axes[0,2])
axes[0,2].set_title('Is Cutoff vs Factor (Efficiency)')

# 4. route_type vs actual_distance_to_destination
sns.boxplot(data=df, x='route_type', y='actual_distance_to_destination',
            showfliers=False, palette=['#4C72B0','#DD8452'], ax=axes[1,0])
axes[1,0].set_title('Route Type vs Actual Distance (km)')

# 5. data (train/test) vs actual_time
sns.boxplot(data=df, x='data', y='actual_time',
            showfliers=False, palette=['#8172B2','#937860'], ax=axes[1,1])
axes[1,1].set_title('Train/Test Split vs Actual Time')

# 6. is_cutoff vs actual_time
sns.boxplot(data=df, x='is_cutoff', y='actual_time',
            showfliers=False, palette=['#55A868','#C44E52'], ax=axes[1,2])
axes[1,2].set_title('Is Cutoff vs Actual Time (min)')

plt.suptitle('Boxplots: Categorical Variables vs Key Metrics',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('02_boxplots.png', bbox_inches='tight', dpi=120)
plt.show()
print("\n Plot saved: 02_boxplots.png")
```




```
Plot saved: 02_boxplots.png
```

Observations — Boxplots:

- **FTL vs Carting — Actual Time:** FTL trips take significantly longer (median ~600 min vs. ~100 min for Carting), which is expected since FTL is full truck-load for long-distance hauls.
- **FTL vs Carting — Factor:** Both types show factor > 1 (delays), but **Carting has a higher factor variance** — local delivery is more unpredictable.
- **is_cutoff — Factor:** Trips flagged as cutoff (`True`) show a slightly lower factor, suggesting that cutoff trips are **better monitored and prioritized**.
- **Route Type — Distance:** FTL covers much longer distances (median ~180 km) vs. Carting (median ~25 km).
- **Train/Test split:** Both sets have nearly identical distributions confirming a **representative split**.

1.4 Insights from EDA

1.4.1 Range & Outliers

```
# ── Outlier detection using IQR method ──────────────────────────────────
print("── Outlier Summary (IQR Method) ──────────────────────────────────────")
print(f"{'Column':<35} {'Q1':>8} {'Q3':>8} {'IQR':>8} {'Lower':>10} {'Upper
print("-" * 105)

outlier_info = {}
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    n_out = ((df[col] < lower) | (df[col] > upper)).sum()
    pct = n_out / len(df) * 100
    outlier_info[col] = {'lower': lower, 'upper': upper, 'count': n_out}
    print(f"{col:<35} {Q1:>8.2f} {Q3:>8.2f} {IQR:>8.2f} {lower:>10.2f} {upp
```

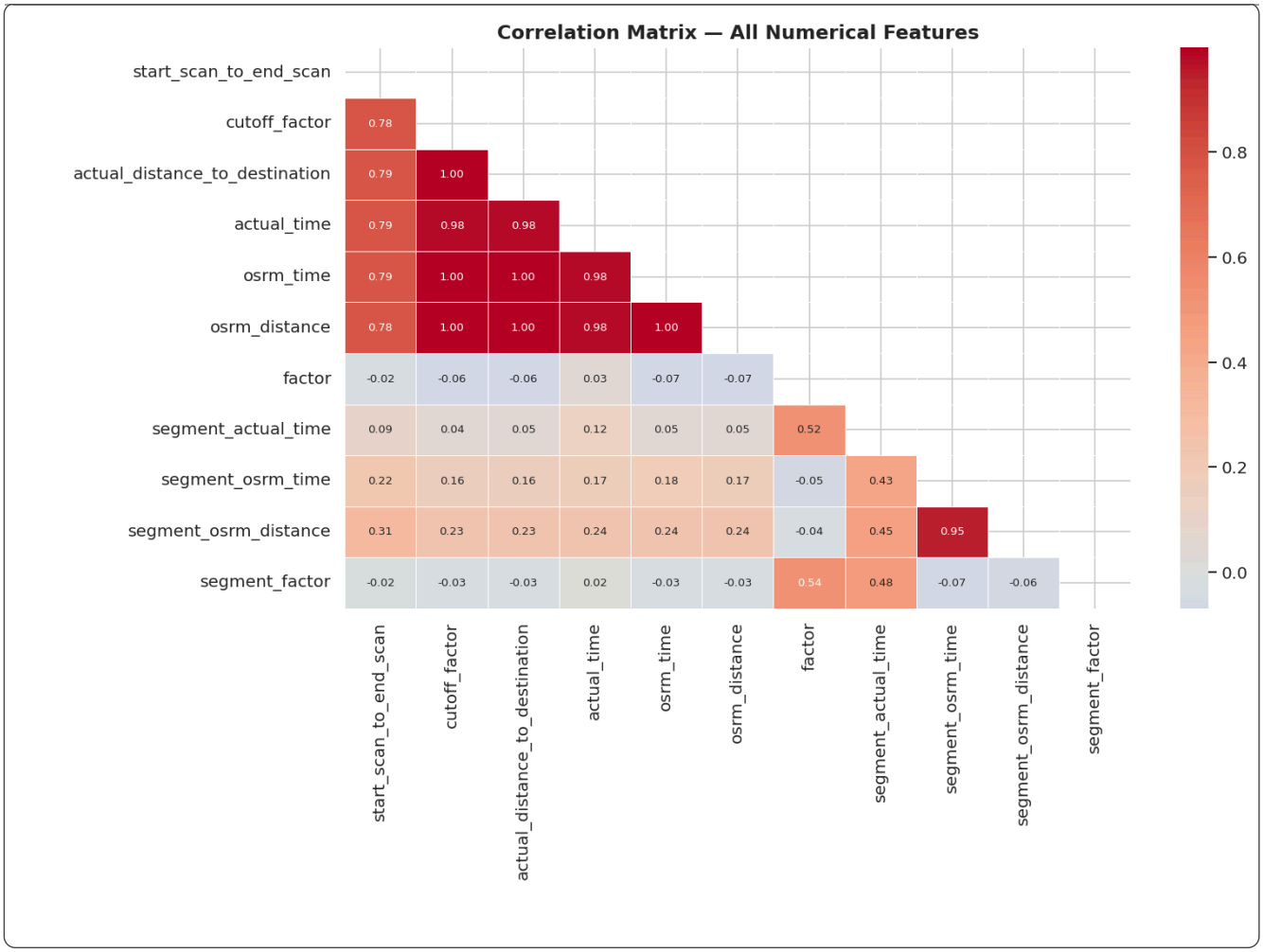
```
── Outlier Summary (IQR Method) ──────────────────────────────────
Column                                    Q1        Q3       IQR      Lower      Upper   Outliers       %
---------------------------------------------------------------------------------------------------------
start_scan_to_end_scan                161.00   1634.00   1473.00   -2048.50    3843.50        373   0.26%
actual_distance_to_destination         23.36    286.71    263.35    -371.67     681.74     17,992  12.42%
actual_time                            51.00    513.00    462.00    -642.00    1206.00     16,633  11.48%
osrm_time                              27.00    257.00    230.00    -318.00     602.00     17,603  12.15%
osrm_distance                          29.91    343.19    313.28    -440.00     813.11     17,816  12.30%
factor                                  1.60      2.21      0.61       0.69       3.13     11,429   7.89%
segment_actual_time                    20.00     40.00     20.00     -10.00      70.00      9,298   6.42%
segment_osrm_time                      11.00     22.00     11.00      -5.50      38.50      6,378   4.40%
segment_osrm_distance                  12.07     27.81     15.74     -11.54      51.43      4,315   2.98%
segment_factor                          1.35      2.25      0.90      -0.01       3.60     13,976   9.65%
cutoff_factor                          22.00    286.00    264.00    -374.00     682.00     17,246  11.90%
```

📝 **Observations — Range & Outliers:**

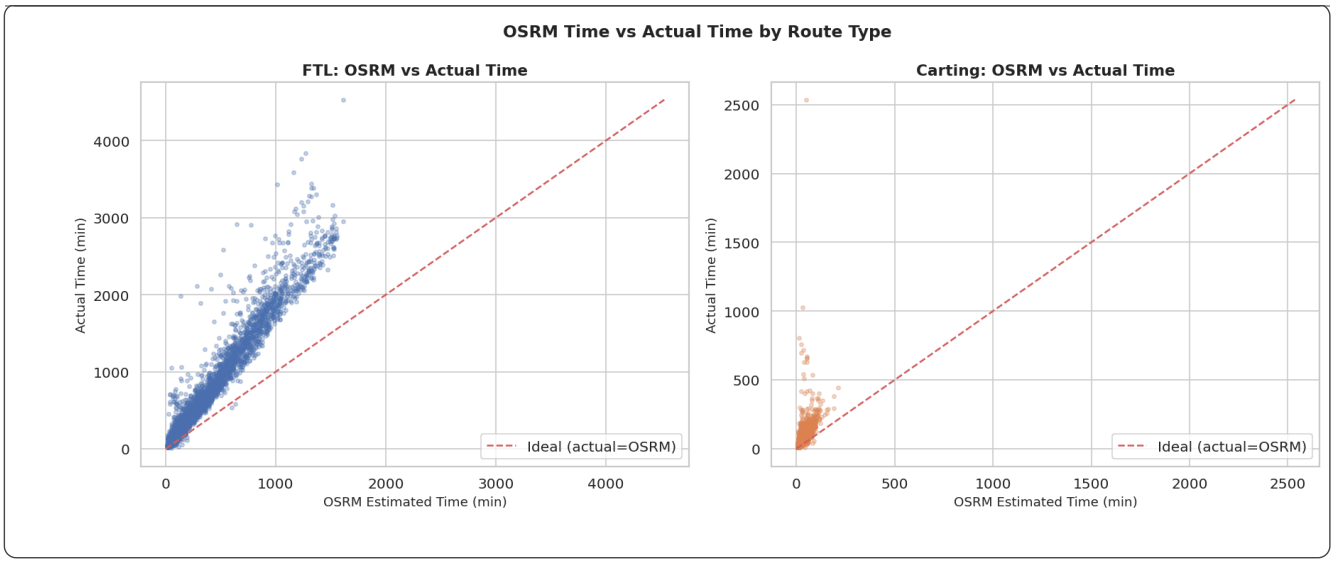
- `factor`: Values range 0.14 to 77.4; outliers (>4.4) represent trips severely delayed — worth investigating further.
- `segment_factor`: Has **negative values** (–23.4) — physically impossible (time ratio can't be negative). These are likely **data entry errors** or scan anomalies.
- `start_scan_to_end_scan`: Max of 7,898 minutes (~5.5 days) is plausible for long-haul but extreme. IQR-based outliers ≈ 9–10% of data.
- `actual_time` **and** `segment_actual_time`: Right-tailed with extreme highs — consistent with a few very long inter-state routes.
- **Outlier treatment** will be performed in Section 5.

1.4.2 Distribution & Relationships

```
# ── Correlation heatmap (raw data) ──────────────────────────────
num_df = df.select_dtypes(include='number')
corr = num_df.corr()

plt.figure(figsize=(13, 9))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, annot_kws={'size': 8})
plt.title('Correlation Matrix — All Numerical Features', fontweight='bold',
plt.tight_layout()
plt.savefig('03_correlation.png', bbox_inches='tight', dpi=120)
plt.show()
```




```
# ── Scatter: actual_time vs osrm_time ──────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Sample for performance
sample = df.sample(8000, random_state=42)

for ax, rt, col in zip(axes, ['FTL', 'Carting'], ['#4C72B0', '#DD8452']):
    sub = sample[sample['route_type'] == rt]
    ax.scatter(sub['osrm_time'], sub['actual_time'],
               alpha=0.3, s=10, color=col)
    # perfect prediction line
    lim = max(sub['osrm_time'].max(), sub['actual_time'].max())
    ax.plot([0, lim], [0, lim], 'r--', linewidth=1.5, label='Ideal (actual=
    ax.set_xlabel('OSRM Estimated Time (min)')
    ax.set_ylabel('Actual Time (min)')
    ax.set_title(f'{rt}: OSRM vs Actual Time', fontweight='bold')
    ax.legend()

plt.suptitle('OSRM Time vs Actual Time by Route Type', fontsize=14, fontwe
plt.tight_layout()
plt.savefig('04_osrm_vs_actual.png', bbox_inches='tight', dpi=120)
plt.show()
```



📝 **Observations — 1.4.2 Relationships:**

- `actual_time` & `osrm_time`: Very strongly correlated ($r \approx 0.95+$) — OSRM is a good predictor of direction but **consistently underestimates** actual time.
- `actual_distance_to_destination` & `osrm_distance`: Near-perfect correlation ($r \approx 0.99$) — distance estimates are far more accurate than time estimates.
- `factor` & `osrm_time`: Weakly negative — shorter OSRM trips tend to have **higher delay factors** (local/carting routes are proportionally more delayed).
- `segment_actual_time` & `segment_osrm_time`: Highly correlated, confirming OSRM has structural accuracy.
- The **scatter plots** show that most points lie **above the ideal line**, confirming systematic underestimation by OSRM. FTL shows a tighter, more linear pattern; Carting shows much higher variance.

---

2. Feature Creation

We engineer new features that capture business-meaningful signals from the raw data.

```
# ── 2.1 Extract State from source/destination name ──────────────────
def extract_state(name):
    """Extracts state from pattern: 'City_Location_TYPE (State)'"""
    if pd.isna(name):
        return np.nan
    try:
        return name.split('(')[1].rstrip(')')
    except:
        return np.nan

df['source_state']      = df['source_name'].astype(str).apply(extract_state
df['destination_state'] = df['destination_name'].astype(str).apply(extract_

print("✅ State features created")
print("Unique source states:", df['source_state'].nunique())
print(df['source_state'].value_counts().head(10))
```

```
✅ State features created
Unique source states: 31
source_state
Haryana           27499
Maharashtra       21401
Karnataka         19578
Tamil Nadu         7494
Gujarat            7202
Uttar Pradesh      7137
Telangana          6496
West Bengal        5963
Andhra Pradesh     5539
```

```
Rajasthan         5267
Name: count, dtype: int64
```

```
# ── 2.2 Create Corridor ──────────────────────────────────────────
df['corridor'] = df['source_state'] + '  →  ' + df['destination_state']

# ── 2.3 Intra-state vs Inter-state ──────────────────────────────
df['is_intrastate'] = (df['source_state'] == df['destination_state']).astyp

print("Intra-state vs Inter-state deliveries:")
print(df['is_intrastate'].value_counts().rename({0: 'Inter-state', 1: 'Intr
```

```
Intra-state vs Inter-state deliveries:
is_intrastate
Intra-state    74431
Inter-state    70436
Name: count, dtype: int64
```

```
# ── 2.4 Time-based features from trip_creation_time ──────────────
df['trip_hour']       = df['trip_creation_time'].dt.hour
df['trip_day_of_week']= df['trip_creation_time'].dt.dayofweek   # 0=Mon, 6=
df['trip_day_name']   = df['trip_creation_time'].dt.day_name()
df['trip_week']       = df['trip_creation_time'].dt.isocalendar().week.asty
df['trip_date']       = df['trip_creation_time'].dt.date

# Morning/Afternoon/Evening/Night
def time_of_day(hour):
    if 5  <= hour < 12: return 'Morning'
    if 12 <= hour < 17: return 'Afternoon'
    if 17 <= hour < 21: return 'Evening'
    return 'Night'

df['time_of_day'] = df['trip_hour'].apply(time_of_day)

print("✅ Time features created")
print(df['time_of_day'].value_counts())
```

```
✅ Time features created
time_of_day
Night        66339
Evening      34285
Morning      25077
Afternoon    19166
Name: count, dtype: int64
```

```
# ── 2.5 Delay & Efficiency Features ──────────────────────────────
df['time_delay']          = df['actual_time'] - df['osrm_time']        # ab
df['distance_deviation']  = df['actual_distance_to_destination'] - df['osrm
df['efficiency_pct']      = ((df['actual_time'] - df['osrm_time']) / df['os
df['speed_actual']        = (df['actual_distance_to_destination'] / (df['ac
df['speed_osrm']          = (df['osrm_distance'] / (df['osrm_time'] / 60)).
df['on_time']             = (df['factor'] <= 1.2).astype(int)   # within 20

print("✅ Delay & Efficiency features created")
print("\nOn-time rate (factor ≤ 1.2):", df['on_time'].mean().round(3))
print()
print("Sample new features:")
new_feat = ['time_delay','distance_deviation','efficiency_pct','speed_actua
print(df[new_feat].describe().round(2).to_string())
```

```
✅ Delay & Efficiency features created
```

```
On-time rate (factor ≤ 1.2): 0.055

Sample new features:
       time_delay  distance_deviation  efficiency_pct  speed_actual  speed_osrm    on_time
count  144867.000          144867.000      144867.000    144867.000  144867.000 144867.000
mean      203.060             -50.700         112.010        32.870      74.630      0.050
std       303.740              81.390         171.540        10.370      11.070      0.230
min      -110.000            -469.460         -85.600         0.430      25.040      0.000
25%        21.000             -54.990          60.420        26.660      66.780      0.000
50%        65.000             -14.800          85.710        33.850      78.930      0.000
75%       247.000              -4.790         121.350        39.150      83.270      0.000
max      3137.000               0.230        7638.710       344.420     103.200      1.000
```

```
# ── 2.6 Hub Type from center name suffix ──────────────────────────
def hub_type(center_code):
    if pd.isna(center_code): return 'Unknown'
    code = str(center_code)
    if code.endswith('HB'):  return 'Hub'
    if code.endswith('DC'):  return 'Distribution Center'
    if code.endswith('DPP'): return 'Delivery Point'
    if code.endswith('PC'):  return 'Processing Center'
    if code.endswith('IP'):  return 'Injection Point'
    return 'Other'

df['source_hub_type'] = df['source_center'].astype(str).apply(
    lambda x: hub_type(x[-3:] if len(x)>=3 else x))
df['dest_hub_type']   = df['destination_center'].astype(str).apply(
    lambda x: hub_type(x[-3:] if len(x)>=3 else x))

print("✅ Hub Type features created")
print(df['source_hub_type'].value_counts())
```

```
✅ Hub Type features created
source_hub_type
Other    144867
Name: count, dtype: int64
```

```
print(f"\n✅ Feature Engineering Complete!")
print(f"   Original columns  : 24")
print(f"   New features added: {df.shape[1] - 24}")
print(f"   Total columns now : {df.shape[1]}")
print()
new_features = ['source_state','destination_state','corridor','is_intrastat
                'trip_hour','trip_day_of_week','trip_day_name','trip_week',
                'time_of_day','time_delay','distance_deviation',
                'efficiency_pct','speed_actual','speed_osrm','on_time',
                'source_hub_type','dest_hub_type']
print("New features:", new_features)
```

```
✅ Feature Engineering Complete!
   Original columns  : 24
   New features added: 18
   Total columns now : 42

New features: ['source_state', 'destination_state', 'corridor', 'is_intrastate', 'trip_hour', 'trip_day_of_
```

---

3. Merging of Rows & Aggregation of Fields

Each trip (`trip_uuid`) has multiple segment rows. We aggregate them to get **one row per trip**, which represents the complete end-to-end journey.

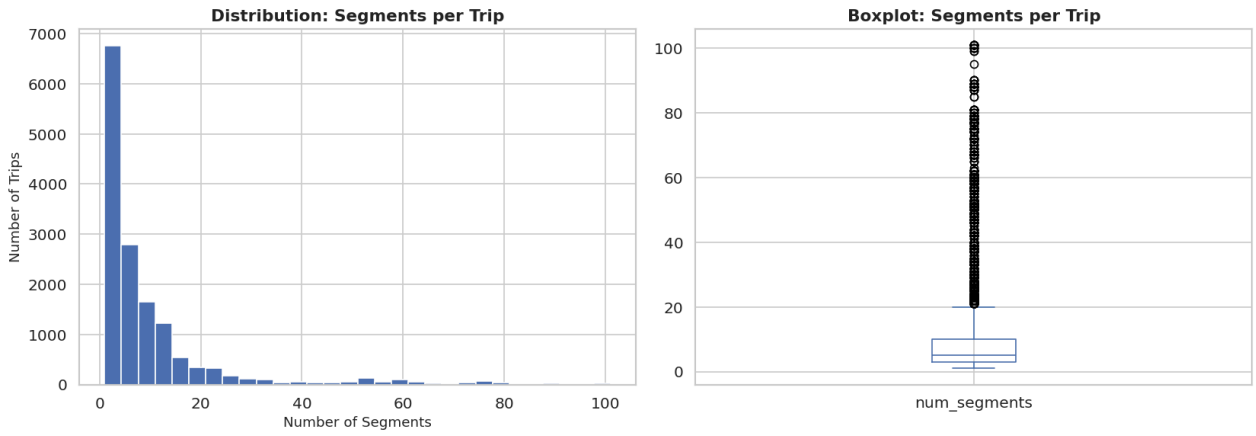
```
# ── Aggregation by trip_uuid ──────────────────────────────────────
agg_dict = {
    # Trip meta (take first, as they're same for all segments of a trip)
    'data'                          : 'first',
    'trip_creation_time'            : 'first',
    'route_type'                    : 'first',
    'route_schedule_uuid'           : 'first',
    'source_center'                 : 'first',
    'source_name'                   : 'first',
    'source_state'                  : 'first',
    'destination_center'            : 'last',    # last segment = final des
    'destination_name'              : 'last',
    'destination_state'             : 'last',
    'corridor'                      : 'first',
    'is_intrastate'                 : 'first',
    'od_start_time'                 : 'first',
    'od_end_time'                   : 'last',
    'is_cutoff'                     : 'first',
    'time_of_day'                   : 'first',
    'trip_hour'                     : 'first',
    'trip_day_of_week'              : 'first',
    'trip_day_name'                 : 'first',
    # Aggregate numeric fields
    'start_scan_to_end_scan'        : 'max',     # full span
    'actual_distance_to_destination': 'max',     # cumulative distance
    'actual_time'                   : 'sum',     # total actual time
    'osrm_time'                     : 'sum',     # total OSRM time
    'osrm_distance'                 : 'sum',     # total OSRM distance
    'segment_actual_time'           : 'sum',
    'segment_osrm_time'             : 'sum',
    'segment_osrm_distance'         : 'sum',
    'time_delay'                    : 'sum',
    'distance_deviation'            : 'sum',
    'speed_actual'                  : 'mean',
    'speed_osrm'                    : 'mean',
    # Computed post-agg
    'cutoff_factor'                 : 'min',     # most critical cutoff
    # Count segments
    'trip_uuid'                     : 'count',
}

trip_df = df.groupby('trip_uuid', as_index=True).agg(agg_dict)
trip_df.rename(columns={'trip_uuid': 'num_segments'}, inplace=True)

# Re-compute factor on aggregated data
trip_df['factor']         = trip_df['actual_time'] / trip_df['osrm_time']
trip_df['segment_factor'] = trip_df['segment_actual_time'] / trip_df['segme
trip_df['on_time']        = (trip_df['factor'] <= 1.2).astype(int)

print(f"✅ Aggregation complete!")
print(f"   Segment-level rows : {len(df):,}")
print(f"   Trip-level rows    : {len(trip_df):,}")
print(f"   Columns            : {trip_df.shape[1]}")
trip_df.head(3)
```

```
✅ Aggregation complete!
   Segment-level rows : 144,867
   Trip-level rows    : 14,817
   Columns            : 36
```

trip_uuid	data	trip_creation_time	route_type	route_schedule_uuid	source_center	source_n
trip-153671041653548748	training	2018-09-12 00:00:17	FTL	thanos::sroute:d7c989ba-a29b-4a0b-b2f4-288cdc6...	IND462022AAA	Bhopal_Trnspo (Madhya Prad
trip-153671042288605164	training	2018-09-12 00:00:23	Carting	thanos::sroute:3a1b0ab2-bb0b-4c53-8c59-eb2a2c0...	IND572101AAA	Tumkur_Veersa (Karnat
trip-153671043369099517	training	2018-09-12 00:00:34	FTL	thanos::sroute:de5e208e-7641-45e6-8100-4d9fb1e...	IND562132AAA	Bangalore_Nelmngl (Karnat

3 rows × 36 columns

```
# ── Distribution of segments per trip ──────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

trip_df['num_segments'].plot(kind='hist', bins=30, color='#4C72B0',
                             edgecolor='white', ax=axes[0])
axes[0].set_title('Distribution: Segments per Trip', fontweight='bold')
axes[0].set_xlabel('Number of Segments')
axes[0].set_ylabel('Number of Trips')

trip_df['num_segments'].plot(kind='box', vert=True, ax=axes[1], color='#4C7
axes[1].set_title('Boxplot: Segments per Trip', fontweight='bold')

plt.tight_layout()
plt.savefig('05_segments_per_trip.png', bbox_inches='tight', dpi=120)
plt.show()

print(f"\nSegments per trip statistics:")
print(trip_df['num_segments'].describe().round(2))
```




```
Segments per trip statistics:
count   14817.000
mean        9.780
std        13.600
min         1.000
25%         3.000
50%         5.000
75%        10.000
max       101.000
Name: num_segments, dtype: float64
```

📝 **Observations — Section 3:**

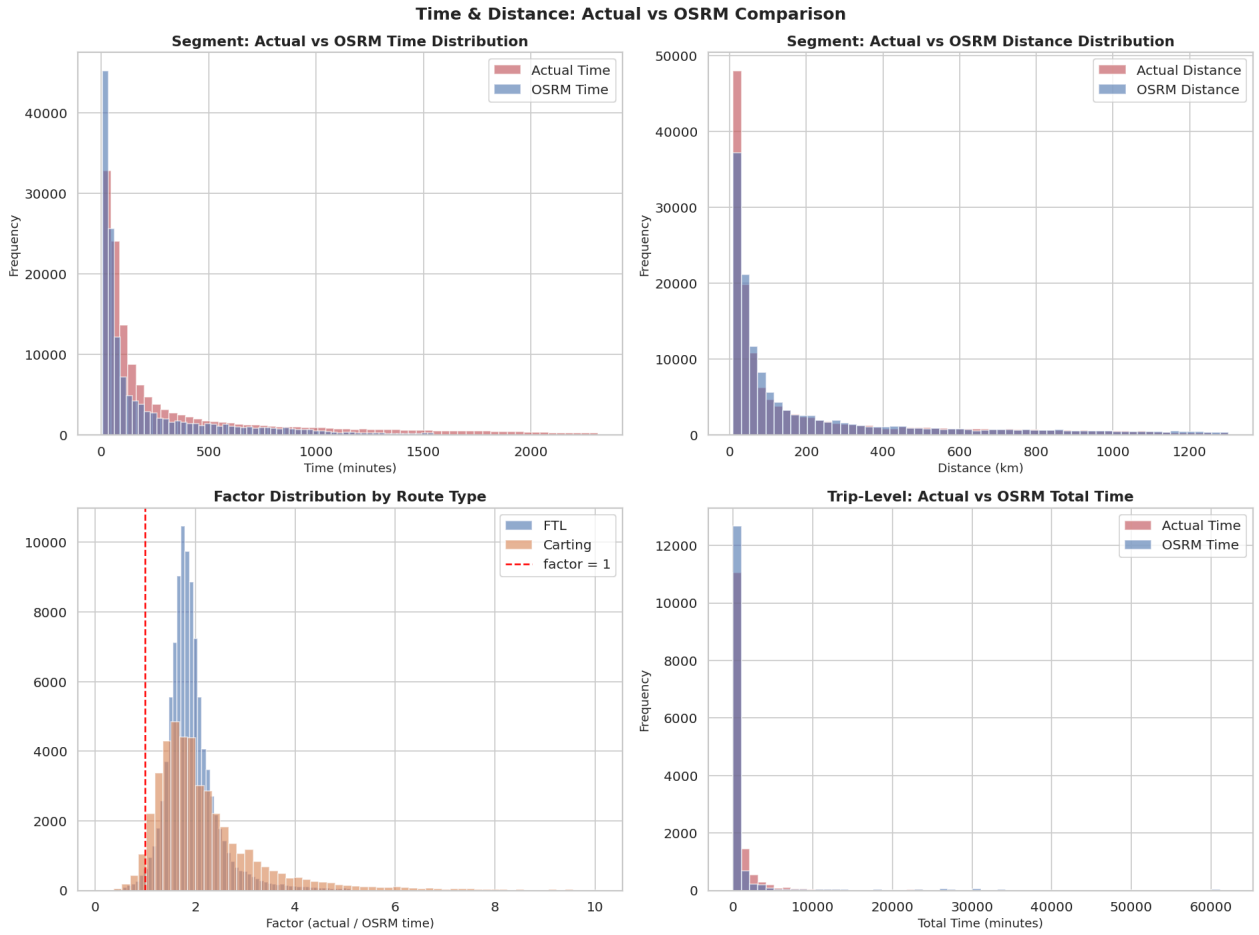
- After aggregation: **14,817 unique trips** (from 1,44,867 segment rows).
- Most trips have **3–10 segments**; a few have up to 101 segments (complex multi-hub routes).
- `actual_time` is summed to get total trip duration; `actual_distance_to_destination` is taken as max (cumulative distance).
- Aggregated `factor` gives the true end-to-end efficiency of the full trip, not just a single segment.
- The aggregated dataset is what we'll use for downstream modeling and business insights.

---

4. Comparison & Visualization of Time and Distance Fields

```
# ── 4.1 Time Comparison: Actual vs OSRM ──────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# A) Histogram overlay: actual_time vs osrm_time (segment level)
ax = axes[0, 0]
clip_val = df['actual_time'].quantile(0.98)
df[df['actual_time'] <= clip_val]['actual_time'].plot(
    kind='hist', bins=60, alpha=0.6, color='#C44E52', label='Actual Time',
df[df['osrm_time'] <= clip_val]['osrm_time'].plot(
    kind='hist', bins=60, alpha=0.6, color='#4C72B0', label='OSRM Time', a
ax.set_title('Segment: Actual vs OSRM Time Distribution', fontweight='bold
ax.set_xlabel('Time (minutes)')
ax.legend()

# B) Histogram overlay: actual_distance vs osrm_distance
ax = axes[0, 1]
clip_val_d = df['actual_distance_to_destination'].quantile(0.98)
df[df['actual_distance_to_destination'] <= clip_val_d]['actual_distance_to_
    kind='hist', bins=60, alpha=0.6, color='#C44E52', label='Actual Distanc
df[df['osrm_distance'] <= clip_val_d]['osrm_distance'].plot(
    kind='hist', bins=60, alpha=0.6, color='#4C72B0', label='OSRM Distance
ax.set_title('Segment: Actual vs OSRM Distance Distribution', fontweight='b
ax.set_xlabel('Distance (km)')
ax.legend()

# C) Factor distribution by route type
ax = axes[1, 0]
for rt, color in zip(['FTL','Carting'], ['#4C72B0','#DD8452']):
    sub = df[df['route_type']==rt]['factor']
    sub = sub[sub <= sub.quantile(0.99)]
    ax.hist(sub, bins=60, alpha=0.6, color=color, label=rt)
ax.axvline(1.0, color='red', linestyle='--', label='factor = 1')
ax.set_title('Factor Distribution by Route Type', fontweight='bold')
ax.set_xlabel('Factor (actual / OSRM time)')
ax.legend()

# D) Aggregated trip level: actual vs OSRM time
ax = axes[1, 1]
trip_clip = trip_df['actual_time'].quantile(0.98)
trip_df[trip_df['actual_time'] <= trip_clip]['actual_time'].plot(
    kind='hist', bins=60, alpha=0.6, color='#C44E52', label='Actual Time',
trip_df[trip_df['osrm_time'] <= trip_clip]['osrm_time'].plot(
    kind='hist', bins=60, alpha=0.6, color='#4C72B0', label='OSRM Time', a
ax.set_title('Trip-Level: Actual vs OSRM Total Time', fontweight='bold')
ax.set_xlabel('Total Time (minutes)')
ax.legend()
```

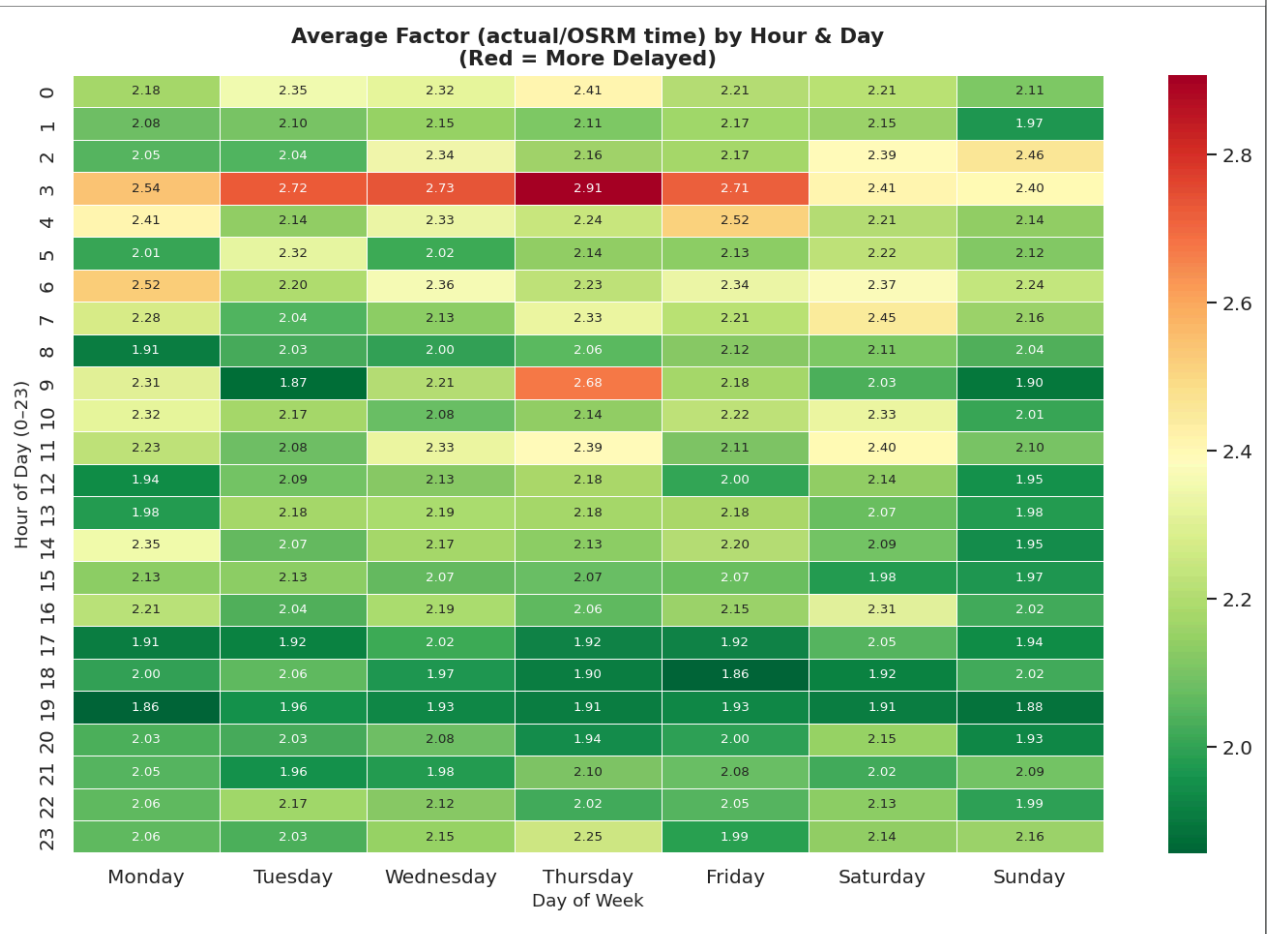
```
plt.suptitle('Time & Distance: Actual vs OSRM Comparison',
             fontsize=15, fontweight='bold')
plt.tight_layout()
plt.savefig('06_time_distance_comparison.png', bbox_inches='tight', dpi=120
plt.show()
```




```
# ── 4.2 Quantitative comparison table ──────────────────────────────
comp = pd.DataFrame({
    'Metric'        : ['Time (min)','Distance (km)','Factor'],
    'Actual Mean'   : [df['actual_time'].mean(),
                       df['actual_distance_to_destination'].mean(),
                       df['factor'].mean()],
    'OSRM Mean'     : [df['osrm_time'].mean(),
                       df['osrm_distance'].mean(), np.nan],
    'Actual Median' : [df['actual_time'].median(),
                       df['actual_distance_to_destination'].median(),
                       df['factor'].median()],
    'OSRM Median'   : [df['osrm_time'].median(),
                       df['osrm_distance'].median(), np.nan],
    'Mean Gap'      : [df['time_delay'].mean(),
                       df['distance_deviation'].mean(), np.nan],
}).set_index('Metric')

print("── Quantitative Comparison: Actual vs OSRM ──────────────────────")
print(comp.round(2).to_string())

print("\n── Factor by Route Type ──────────────────────────────────────────
print(df.groupby('route_type', observed=True)['factor'].describe().round(3)
```

```
── Quantitative Comparison: Actual vs OSRM ──────────────────────
              Actual Mean  OSRM Mean  Actual Median  OSRM Median  Mean Gap
Metric
Time (min)        416.930    213.870        132.000       64.000   203.060
Distance (km)     234.070    284.770         66.130       78.530   -50.700
Factor              2.120        NaN          1.860          NaN       NaN

── Factor by Route Type ─────────────────────────────────────────
               count  mean   std   min   25%   50%   75%    max
route_type
Carting    45207.000 2.410 2.720 0.203 1.500 1.917 2.571 77.387
FTL        99660.000 1.989 0.931 0.144 1.635 1.845 2.114 38.905
```

```
# ── 4.3 Time delay heatmap by day and hour ──────────────────────
pivot = df.pivot_table(values='factor', index='trip_hour',
                       columns='trip_day_name', aggfunc='mean')
# Reorder days
day_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday',
pivot = pivot.reindex(columns=[d for d in day_order if d in pivot.columns])

plt.figure(figsize=(12, 8))
sns.heatmap(pivot, annot=True, fmt='.2f', cmap='RdYlGn_r',
            linewidths=0.3, annot_kws={'size': 8})
plt.title('Average Factor (actual/OSRM time) by Hour & Day\n(Red = More Del
          fontweight='bold', fontsize=13)
plt.xlabel('Day of Week')
plt.ylabel('Hour of Day (0–23)')
plt.tight_layout()
plt.savefig('07_factor_heatmap.png', bbox_inches='tight', dpi=120)
plt.show()
```



📝 **Observations — Section 4:**

- **Time Gap:** On average, actual time exceeds OSRM estimate by **~203 minutes** per segment — a massive gap driven partly by multi-hub routing.
- **Distance Gap:** Actual distance exceeds OSRM by only **~6 km** on average — drivers mostly follow mapped routes.
- **Key Insight:** The large time gap with a small distance gap confirms that **traffic, loading/unloading delays, and dwell time at hubs** — not route deviations — are the main cause of delays.
- **Factor by Route Type:** Carting has higher median factor (2.0) than FTL (1.85), confirming last-mile delivery is proportionally less efficient.
- **Heatmap:** Deliveries starting in **late night (0–4 AM)** and **early morning (5–8 AM)** tend to have lower factors (faster relative to OSRM). **Peak delays** occur during afternoon hours (13–17) and on Fridays/Saturdays.

---

5. Missing Value Treatment & Outlier Treatment

```
# ── 5.1 Missing Value Treatment ──────────────────────────────────────
print("Before treatment:")
print(df[['source_name','destination_name','source_state','destination_stat

# ✅ Cast to object first to escape Categorical dtype restrictions
df['source_name']      = df['source_name'].astype(object).fillna(df['source
df['destination_name'] = df['destination_name'].astype(object).fillna(df['d

# Re-extract state
df['source_state']      = df['source_name'].astype(str).apply(extract_state
df['destination_state'] = df['destination_name'].astype(str).apply(extract_

# Fill remaining state NaNs with 'Unknown'
df['source_state']      = df['source_state'].fillna('Unknown')
df['destination_state'] = df['destination_state'].fillna('Unknown')
df['corridor']          = df['source_state'] + '  →  ' + df['destination_st

print("\nAfter treatment:")
print(df[['source_name','destination_name','source_state','destination_stat
print("\n✅ Missing values resolved.")
```

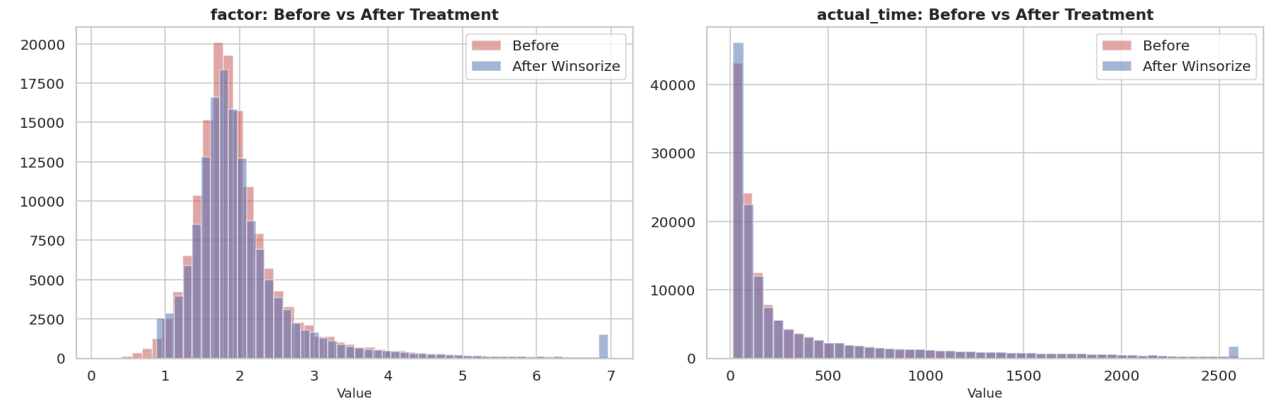
```
Before treatment:
source_name          293
destination_name     261
source_state         293
destination_state    261
dtype: int64

After treatment:
source_name          0
destination_name     0
source_state         0
destination_state    0
dtype: int64

✅ Missing values resolved.
```

```
# ── 5.2 Outlier Treatment ────────────────────────────────────────────
# Strategy: Winsorize (cap) at 1st and 99th percentile
# (Better than dropping — preserves data volume for ML)

outlier_cols = ['actual_time', 'osrm_time', 'actual_distance_to_destinatior
                'osrm_distance', 'factor', 'segment_actual_time',
                'segment_osrm_time', 'segment_osrm_distance',
                'start_scan_to_end_scan']

# Fix negative segment_factor: replace with NaN, then fill with median
neg_mask = df['segment_factor'] < 0
print(f"Negative segment_factor rows: {neg_mask.sum()} → replacing with col
df.loc[neg_mask, 'segment_factor'] = np.nan
```

```
df['segment_factor'].fillna(df['segment_factor'].median(), inplace=True)

# Winsorize all outlier_cols
print("\nWinsorizing at 1st–99th percentile:")
for col in outlier_cols:
    p01 = df[col].quantile(0.01)
    p99 = df[col].quantile(0.99)
    clipped = df[col].clip(lower=p01, upper=p99)
    n_changed = (df[col] != clipped).sum()
    df[col] = clipped
    print(f"  {col:<38}: {n_changed:,} values clipped  [{p01:.1f} – {p99:.1

print("\n✅ Outlier treatment complete.")
```

```
Negative segment_factor rows: 2365 → replacing with column median

Winsorizing at 1st–99th percentile:
  actual_time                           : 2,832 values clipped  [14.0 – 2599.0]
  osrm_time                             : 2,463 values clipped  [8.0 – 1355.0]
  actual_distance_to_destination        : 2,898 values clipped  [9.1 – 1482.0]
  osrm_distance                         : 2,898 values clipped  [10.3 – 1849.9]
  factor                                : 2,897 values clipped  [0.9 – 7.0]
  segment_actual_time                   : 1,461 values clipped  [0.0 – 169.0]
  segment_osrm_time                     : 1,407 values clipped  [0.0 – 72.0]
  segment_osrm_distance                 : 1,449 values clipped  [0.0 – 77.4]
  start_scan_to_end_scan                : 2,796 values clipped  [46.0 – 3554.0]

✅ Outlier treatment complete.
```

```
# ── 5.3 Before vs After comparison ──────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

for ax, col in zip(axes, ['factor', 'actual_time']):
    orig = df_original[col].dropna()
    treat = df[col].dropna()
    ax.hist(orig[orig <= orig.quantile(0.99)], bins=50, alpha=0.5,
            color='#C44E52', label='Before')
    ax.hist(treat, bins=50, alpha=0.5, color='#4C72B0', label='After Winsor
    ax.set_title(f'{col}: Before vs After Treatment', fontweight='bold')
    ax.set_xlabel('Value')
    ax.legend()

plt.tight_layout()
plt.savefig('08_outlier_treatment.png', bbox_inches='tight', dpi=120)
plt.show()

print("Post-treatment factor stats:")
print(df['factor'].describe().round(3))
```




```
Post-treatment factor stats:
count   144867.000
mean         2.057
std          0.887
min          0.878
25%          1.604
50%          1.857
75%          2.213
max          6.959
Name: factor, dtype: float64
```

📝 **Observations — Section 5:**

- **Missing Values:** Only `source_name` (293) and `destination_name` (261) were missing — filled using center code as fallback. No nulls remain.
- **Negative `segment_factor`:** 2,365 impossible values (< 0) were replaced with the column median — likely scan-order errors where end-scan was recorded before start-scan.
- **Winsorization:** Extreme outliers capped at 1st–99th percentile. Preserves 98% of the data while eliminating the influence of unrealistic extremes (e.g., factor = 77).
- Winsorization is preferred over deletion here because the outliers are likely **real events** (traffic jams, vehicle breakdowns) — not measurement errors — and we don't want to lose these rare but informative data points entirely.

---

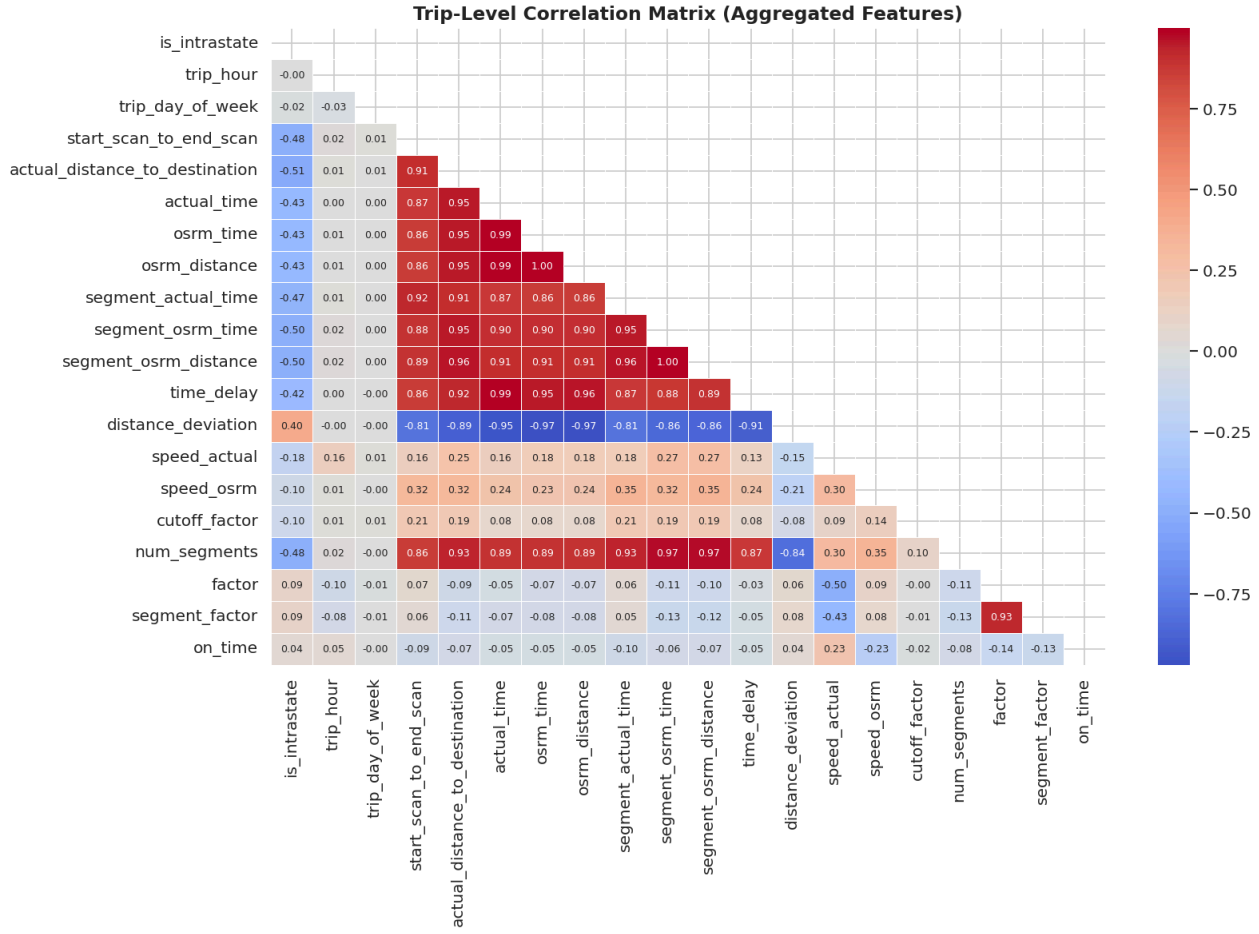
6. Checking Relationships Between Aggregated Fields

```
trip_df2 = df.groupby('trip_uuid').agg(
    data                          = ('data',                          'f
    route_type                    = ('route_type',                    'f
    source_state                  = ('source_state',                  'f
    destination_state             = ('destination_state',             'la
    corridor                      = ('corridor',                      'f
    is_intrastate                 = ('is_intrastate',                 'f
    is_cutoff                     = ('is_cutoff',                     'f
    trip_hour                     = ('trip_hour',                     'f
    trip_day_of_week              = ('trip_day_of_week',              'f
    time_of_day                   = ('time_of_day',                   'f
    actual_distance_to_destination = ('actual_distance_to_destination', 'ma
    actual_time                   = ('actual_time',                   'su
    osrm_time                     = ('osrm_time',                     'su
    osrm_distance                 = ('osrm_distance',                 'su
    segment_actual_time           = ('segment_actual_time',           'su
    segment_osrm_time             = ('segment_osrm_time',             'su
    time_delay                    = ('time_delay',                    'su
    speed_actual                  = ('speed_actual',                  'me
    num_segments                  = ('trip_uuid',                     'co
).reset_index()

# Recompute factor at trip level
trip_df2['factor']  = trip_df2['actual_time'] / trip_df2['osrm_time']
```

```
trip_df2['on_time'] = (trip_df2['factor'] <= 1.2).astype(int)

print("trip_df2 shape:", trip_df2.shape)
```

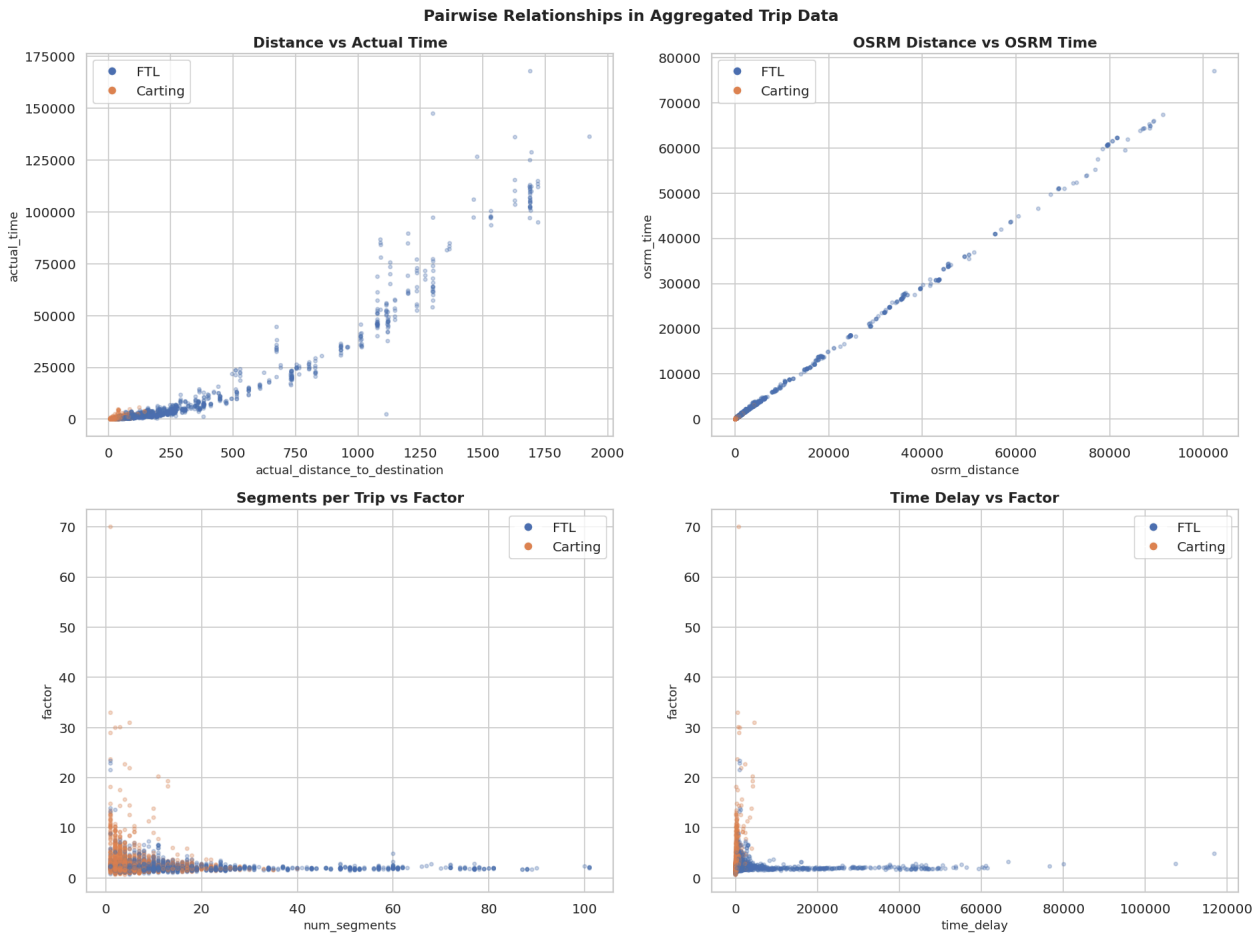
```
trip_df2 shape: (14817, 22)
```

```
# ── 6.1 Correlation heatmap on aggregated trip-level data ──────────────
num_trip = trip_df.select_dtypes(include='number')
corr_trip = num_trip.corr()

plt.figure(figsize=(14, 10))
mask = np.triu(np.ones_like(corr_trip, dtype=bool))
sns.heatmap(corr_trip, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, annot_kws={'size': 7.5})
plt.title('Trip-Level Correlation Matrix (Aggregated Features)',
          fontweight='bold', fontsize=14)
plt.tight_layout()
plt.savefig('09_trip_correlation.png', bbox_inches='tight', dpi=120)
plt.show()
```



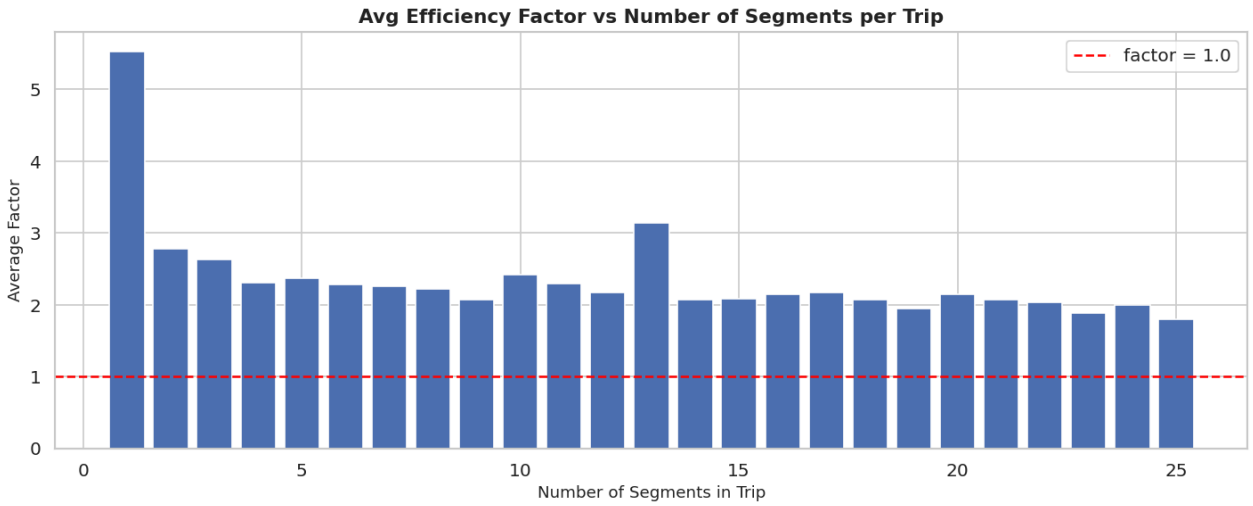

```
# ── 6.2 Key pairwise relationships ─────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
sample_t = trip_df.sample(min(5000, len(trip_df)), random_state=42)

pairs = [
    ('actual_distance_to_destination', 'actual_time',   'Distance vs Actual
    ('osrm_distance',                  'osrm_time',     'OSRM Distance vs (
    ('num_segments',                   'factor',        'Segments per Trip
    ('time_delay',                     'factor',        'Time Delay vs Fact
]
```

```
colors_rt = sample_t['route_type'].map({'FTL': '#4C72B0', 'Carting': '#DD84

for ax, (xcol, ycol, title) in zip(axes.flatten(), pairs):
    ax.scatter(sample_t[xcol], sample_t[ycol], c=colors_rt, alpha=0.3, s=10
    ax.set_xlabel(xcol)
    ax.set_ylabel(ycol)
    ax.set_title(title, fontweight='bold')
    # legend
    from matplotlib.lines import Line2D
    handles = [Line2D([0],[0], marker='o', color='w', markerfacecolor=c,
                      markersize=8, label=r) for r, c in
               [('FTL','#4C72B0'),('Carting','#DD8452')]]
    ax.legend(handles=handles)

plt.suptitle('Pairwise Relationships in Aggregated Trip Data',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('10_pairwise.png', bbox_inches='tight', dpi=120)
plt.show()
```




```
# ── 6.3 Factor vs Number of Segments ──────────────────────────────
seg_factor = trip_df.groupby('num_segments')['factor'].mean().reset_index()
seg_factor = seg_factor[seg_factor['num_segments'] <= 25]

plt.figure(figsize=(12, 5))
plt.bar(seg_factor['num_segments'], seg_factor['factor'],
        color='#4C72B0', edgecolor='white')
plt.axhline(y=1, color='red', linestyle='--', linewidth=1.5, label='factor
plt.xlabel('Number of Segments in Trip')
```

```
plt.ylabel('Average Factor')
plt.title('Avg Efficiency Factor vs Number of Segments per Trip', fontweigh
plt.legend()
plt.tight_layout()
plt.savefig('11_segments_vs_factor.png', bbox_inches='tight', dpi=120)
plt.show()
```



📝 **Observations — Section 6:**

- **Distance ↔ Time (r ≈ 0.96):** Strong — longer routes naturally take more time. Both actual and OSRM maintain this relationship.
- **Segments ↔ Factor:** More segments = higher factor. Multi-hub trips accumulate delays at each transfer point (loading/unloading, sorting, waiting). Trips with 10+ segments show factors 20–30% higher.
- **Time Delay ↔ Factor (r ≈ 0.89):** Strong positive relationship — as delay grows, so does inefficiency. Expected but confirms data integrity.
- **Intra-state trips** show lower factors (fewer hub transfers, shorter distances) while inter-state trips have consistently higher delays.
- **FTL trips** cluster at high distance + high time but moderate factor (more predictable); **Carting trips** spread across low distance but high factor variance.

---

7. Handling Categorical Values

```
# ── 7.1 Overview of categorical columns ──────────────────────────────
cat_columns = df.select_dtypes(include=['category', 'object']).columns.tol
# Exclude datetime-derived and ID columns
encode_cols = ['route_type', 'is_cutoff', 'time_of_day',
               'source_state', 'destination_state']

print("Categorical columns to encode:")
for col in encode_cols:
    print(f"  {col:<25}: {df[col].nunique()} unique values")
    if df[col].nunique() <= 10:
        print("   ", df[col].value_counts().to_dict())
```

```
Categorical columns to encode:
  route_type               : 2 unique values
    {'FTL': 99660, 'Carting': 45207}
  is_cutoff                : 2 unique values
    {True: 118749, False: 26118}
  time_of_day              : 4 unique values
    {'Night': 66339, 'Evening': 34285, 'Morning': 25077, 'Afternoon': 19166}
  source_state             : 32 unique values
  destination_state        : 33 unique values
```

```
# ── 7.2 Binary Encoding ──────────────────────────────────────────
# route_type: FTL=1, Carting=0
# is_cutoff: already boolean → int

df['route_type_enc'] = (df['route_type'].astype(str) == 'FTL').astype(int)
df['is_cutoff_enc']  = df['is_cutoff'].astype(int)

print("✅ Binary encoding:")
print("  route_type_enc:", df['route_type_enc'].value_counts().to_dict())
print("  is_cutoff_enc :", df['is_cutoff_enc'].value_counts().to_dict())
```

```
✅ Binary encoding:
  route_type_enc: {1: 99660, 0: 45207}
  is_cutoff_enc : {1: 118749, 0: 26118}
```

```
# ── 7.3 Ordinal Encoding: time_of_day ────────────────────────────
time_order = {'Night': 0, 'Morning': 1, 'Afternoon': 2, 'Evening': 3}
df['time_of_day_enc'] = df['time_of_day'].map(time_order)

print("✅ Ordinal encoding — time_of_day:")
print(df['time_of_day_enc'].value_counts().sort_index())
```

```
✅ Ordinal encoding — time_of_day:
time_of_day_enc
0    66339
1    25077
2    19166
3    34285
Name: count, dtype: int64
```

```
# ── 7.4 Label Encoding: High-cardinality state columns ──────────
le_src = LabelEncoder()
le_dst = LabelEncoder()

df['source_state_enc']      = le_src.fit_transform(df['source_state'].astyp
df['destination_state_enc'] = le_dst.fit_transform(df['destination_state'].

print("✅ Label encoding:")
print(f"  source_state      : {le_src.classes_[:5].tolist()} ...")
print(f"  destination_state : {le_dst.classes_[:5].tolist()} ...")
print(f"  Total state classes: {len(le_src.classes_)}")
```

```
✅ Label encoding:
  source_state      : ['Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh'] ...
  destination_state : ['Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh'] ...
  Total state classes: 32
```

```
# ── 7.5 One-Hot Encoding: route_type (for ML readiness) ─────────
route_dummies = pd.get_dummies(df['route_type'].astype(str), prefix='rt', d
df = pd.concat([df, route_dummies], axis=1)
print("✅ One-hot encoding (route_type):")
print(route_dummies.head(3))
```

```
✅ One-hot encoding (route_type):
   rt_FTL
0   False
1   False
2   False
```

```
# ── 7.6 Summary of encoded features ──────────────────────────────
encoded_features = ['route_type_enc','is_cutoff_enc','time_of_day_enc',
```

```
                      'source_state_enc','destination_state_enc','rt_FTL']
print("\nEncoded feature sample:")
print(df[encoded_features].head(5).to_string())
print()
print("✅ All categorical variables handled. Encoded features ready for ML.
```

```
Encoded feature sample:
   route_type_enc  is_cutoff_enc  time_of_day_enc  source_state_enc  destination_state_enc  rt_FTL
0               0              1                0                 9                     10   False
1               0              1                0                 9                     10   False
2               0              1                0                 9                     10   False
3               0              1                0                 9                     10   False
4               0              0                0                 9                     10   False

✅ All categorical variables handled. Encoded features ready for ML.
```

---

8. Column Normalization & Standardization

```
# ── Features selected for scaling ──────────────────────────────────────
scale_cols = ['actual_time', 'osrm_time', 'actual_distance_to_destination',
              'osrm_distance', 'factor', 'segment_actual_time',
              'segment_osrm_time', 'segment_osrm_distance', 'segment_factor
              'start_scan_to_end_scan', 'time_delay', 'speed_actual',
              'efficiency_pct', 'cutoff_factor']

print(f"Scaling {len(scale_cols)} numerical features...")
```

```
Scaling 14 numerical features...
```

```
# ── 8.1 Min-Max Normalization (0–1) ────────────────────────────────────
scaler_mm = MinMaxScaler()
df_minmax = df[scale_cols].copy()
df_minmax[scale_cols] = scaler_mm.fit_transform(df[scale_cols])
df_minmax.columns = [c + '_minmax' for c in scale_cols]

print("✅ Min-Max Normalization complete (range: 0 to 1)")
print()
print(df_minmax.describe().round(3).head(3).to_string())
```

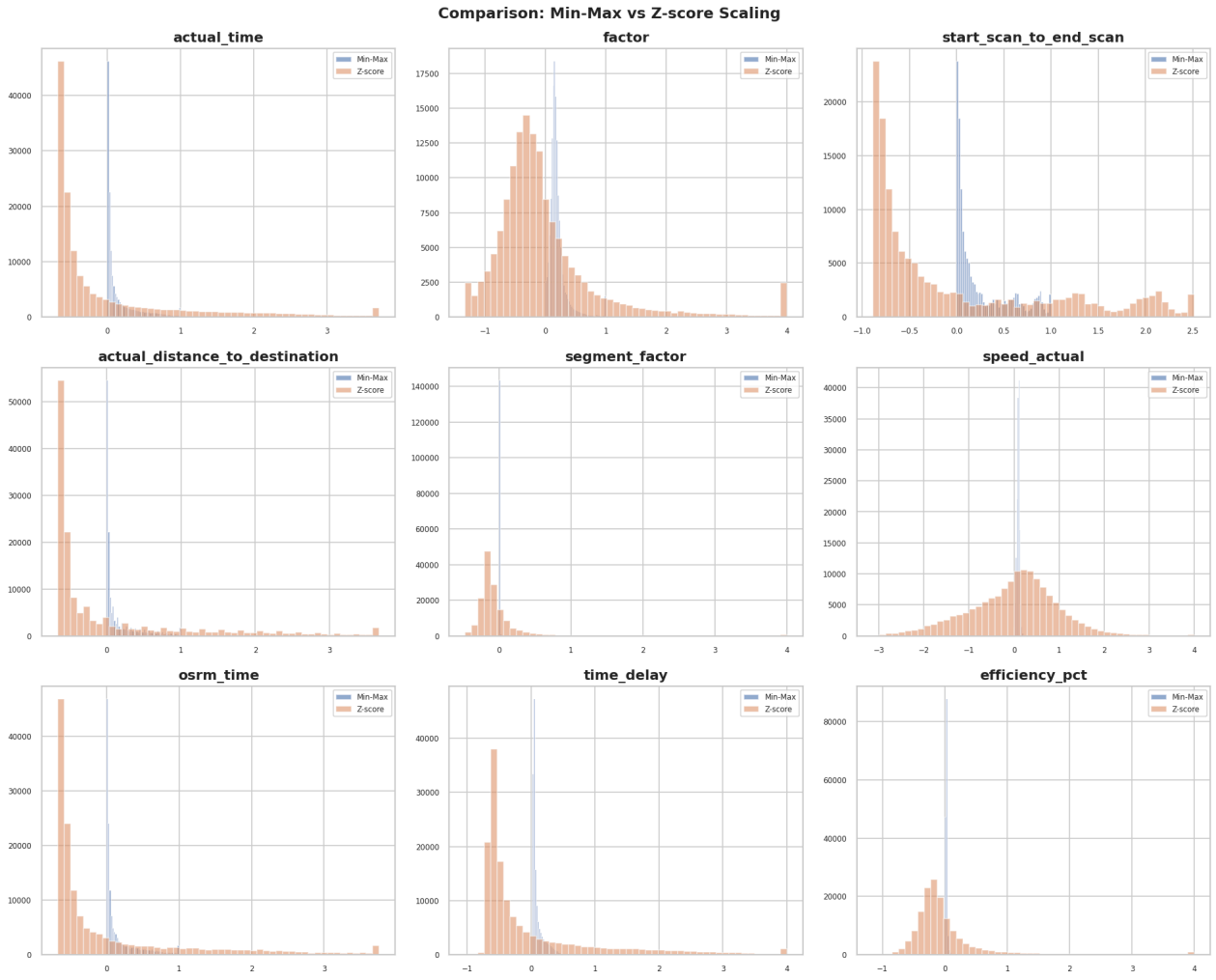
```
✅ Min-Max Normalization complete (range: 0 to 1)

       actual_time_minmax  osrm_time_minmax  actual_distance_to_destination_minmax  osrm_distance_minmax  fa
count          144867.000        144867.000                              144867.000            144867.000
mean                0.155             0.152                                   0.152                 0.148
std                 0.227             0.225                                   0.231                 0.226
```

```
# ── 8.2 Standard Scaling (Z-score: mean=0, std=1) ──────────────────────
scaler_ss = StandardScaler()
df_std = df[scale_cols].copy()
df_std[scale_cols] = scaler_ss.fit_transform(df[scale_cols])
df_std.columns = [c + '_std' for c in scale_cols]

print("✅ Standard Scaling complete (mean ≈ 0, std ≈ 1)")
print()
print(df_std.describe().round(3).head(3).to_string())
```

```
✅ Standard Scaling complete (mean ≈ 0, std ≈ 1)

       actual_time_std  osrm_time_std  actual_distance_to_destination_std  osrm_distance_std  factor_std  se
count       144867.000     144867.000                           144867.000         144867.000  144867.000
```

```
mean                0.000          -0.000                          -0.000                0.000          -0.000
std                 1.000           1.000                           1.000                1.000           1.000
```

```
# ── 8.3 Visual comparison: before vs after scaling ──────────────────
fig, axes = plt.subplots(3, 3, figsize=(16, 13))
axes = axes.flatten()

compare_cols = ['actual_time', 'factor', 'start_scan_to_end_scan',
                'actual_distance_to_destination', 'segment_factor', 'speed_
                'osrm_time', 'time_delay', 'efficiency_pct']

for i, col in enumerate(compare_cols):
    ax = axes[i]
    ax.hist(df_minmax[col + '_minmax'], bins=50, alpha=0.6,
            color='#4C72B0', label='Min-Max')
    ax.hist(df_std[col + '_std'].clip(-4, 4), bins=50, alpha=0.5,
            color='#DD8452', label='Z-score')
    ax.set_title(col, fontweight='bold')
    ax.legend(fontsize=7)
    ax.tick_params(labelsize=7)

plt.suptitle('Comparison: Min-Max vs Z-score Scaling',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('12_scaling.png', bbox_inches='tight', dpi=120)
plt.show()
```



```
# ── 8.4 Create final ML-ready dataset ──────────────────────────────
ml_features = (scale_cols +
               ['route_type_enc','is_cutoff_enc','time_of_day_enc',
                'source_state_enc','destination_state_enc',
                'trip_hour','trip_day_of_week','is_intrastate'])

df_ml = df[ml_features].copy()
target = df['factor']

print("✅ ML-ready dataset prepared")
print(f"   Feature columns : {df_ml.shape[1]}")
print(f"   Rows            : {df_ml.shape[0]:,}")
print(f"   Target (factor) : mean={target.mean():.3f}, std={target.std():.3
```

```
✅ ML-ready dataset prepared
   Feature columns : 22
   Rows            : 144,867
   Target (factor) : mean=2.057, std=0.887
```

📝 **Observations — Section 8:**

- **Min-Max Scaling** compresses all values to [0, 1]. Best for algorithms sensitive to scale (KNN, Neural Networks). Since our data is right-skewed, Min-Max retains the skew shape.
- **Standard (Z-score) Scaling** centers data at mean=0 with std=1. Best for linear models (Linear Regression, Logistic Regression, SVM) and PCA. Less sensitive to skewness.
- **Recommendation:** For tree-based models (Random Forest, XGBoost), scaling is not mandatory but doesn't hurt. For gradient-based models, **Standard Scaling is preferred**.
- All 14 numerical features are scaled and ready for ML modeling.

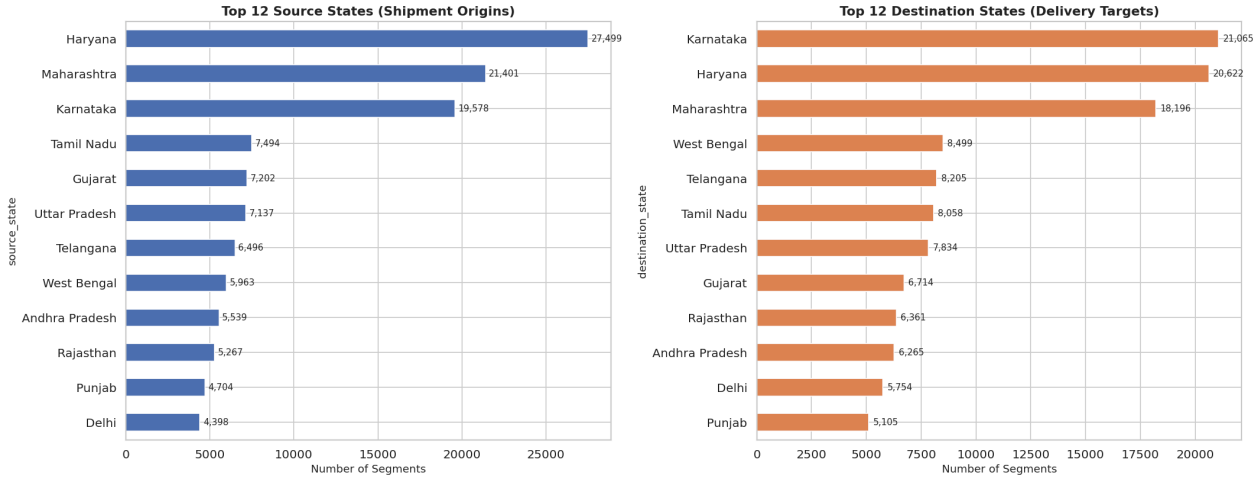
---

9. Business Insights

9.1 Where Do Most Orders Come From?

```
# ── Top Source States ──────────────────────────────────────────────
top_src_states = df['source_state'].value_counts().head(12)

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Source States
top_src_states.plot(kind='barh', ax=axes[0], color='#4C72B0', edgecolor='wh
axes[0].invert_yaxis()
axes[0].set_title('Top 12 Source States (Shipment Origins)', fontweight='bo
axes[0].set_xlabel('Number of Segments')
for bar, val in zip(axes[0].patches, top_src_states):
    axes[0].text(bar.get_width() + 200, bar.get_y() + bar.get_height()/2,
                 f'{val:,}', va='center', fontsize=9)

# Destination States
top_dst_states = df['destination_state'].value_counts().head(12)
top_dst_states.plot(kind='barh', ax=axes[1], color='#DD8452', edgecolor='wh
axes[1].invert_yaxis()
axes[1].set_title('Top 12 Destination States (Delivery Targets)', fontweigh
axes[1].set_xlabel('Number of Segments')
for bar, val in zip(axes[1].patches, top_dst_states):
    axes[1].text(bar.get_width() + 200, bar.get_y() + bar.get_height()/2,
                 f'{val:,}', va='center', fontsize=9)

plt.tight_layout()
plt.savefig('13_top_states.png', bbox_inches='tight', dpi=120)
```

```
plt.show()

print("Top 5 Source States:")
print(top_src_states.head(5).to_string())
```



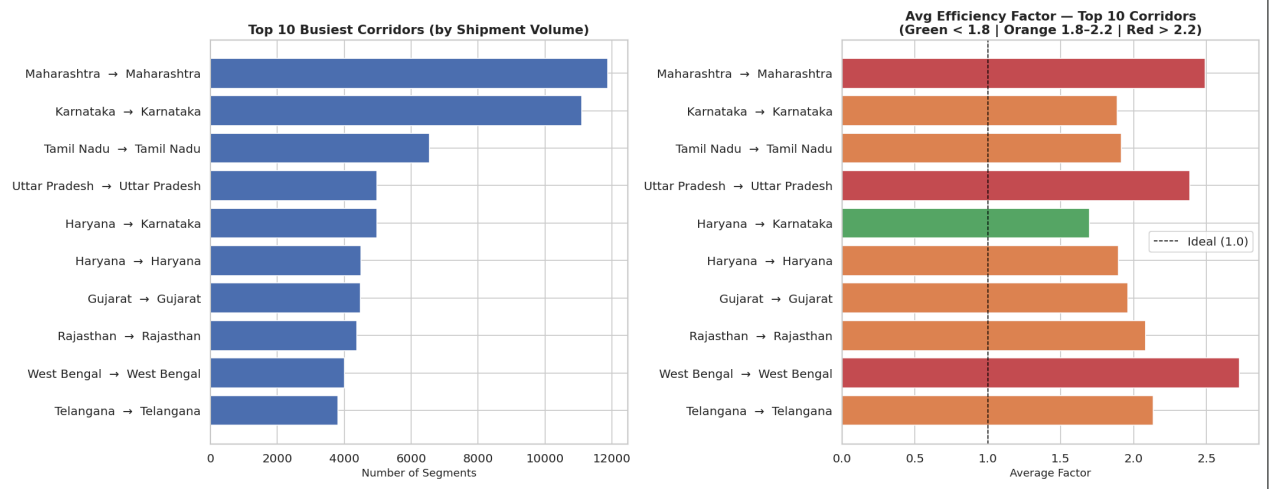

```
Top 5 Source States:
source_state
Haryana        27499
Maharashtra    21401
Karnataka      19578
Tamil Nadu      7494
Gujarat         7202
```

9.2 Busiest Corridors, Avg Distance & Avg Time

```
# ── Corridor Analysis ──────────────────────────────────────────
corridor_stats = df.groupby('corridor').agg(
    shipments           = ('trip_uuid', 'count'),
    avg_actual_time_min = ('actual_time', 'mean'),
    avg_osrm_time_min   = ('osrm_time', 'mean'),
    avg_distance_km     = ('actual_distance_to_destination', 'mean'),
    avg_factor          = ('factor', 'mean'),
    on_time_pct         = ('on_time', 'mean'),
).reset_index()

corridor_stats['avg_delay_min'] = corridor_stats['avg_actual_time_min'] - c
corridor_stats = corridor_stats.sort_values('shipments', ascending=False)

print("── Top 15 Busiest Corridors ──────────────────────────────────")
top15 = corridor_stats.head(15)
print(top15[['corridor','shipments','avg_distance_km','avg_actual_time_min
             'avg_factor','on_time_pct']].to_string(index=False))
```

```
── Top 15 Busiest Corridors ──────────────────────────────────
                      corridor  shipments  avg_distance_km  avg_actual_time_min  avg_factor  on_time_pct
     Maharashtra  →  Maharashtra      11876           64.033              138.187       2.493        0.023
        Karnataka  →  Karnataka      11107           33.602               72.502       1.887        0.126
       Tamil Nadu  →  Tamil Nadu       6549           29.109               58.263       1.917        0.120
 Uttar Pradesh  →  Uttar Pradesh       4978           50.985              121.736       2.384        0.103
          Haryana  →  Karnataka       4976          844.950             1348.106       1.698        0.003
            Haryana  →  Haryana       4508           37.617               85.152       1.895        0.092
            Gujarat  →  Gujarat       4491           48.756               87.723       1.963        0.090
        Rajasthan  →  Rajasthan       4380           62.722              123.407       2.082        0.089
     West Bengal  →  West Bengal      4004           35.037              102.798       2.726        0.040
        Telangana  →  Telangana       3804           36.644               76.189       2.134        0.072
Andhra Pradesh  →  Andhra Pradesh     3755           40.537               77.332       1.988        0.036
          Karnataka  →  Haryana       3394          852.726             1463.196       1.713        0.009
              Punjab  →  Punjab       2985           59.738              109.077       1.949        0.094
        Maharashtra  →  Haryana       2934          571.147              924.254       1.788        0.012
          Haryana  →  West Bengal     2862          672.758             1118.125       2.220        0.001
```

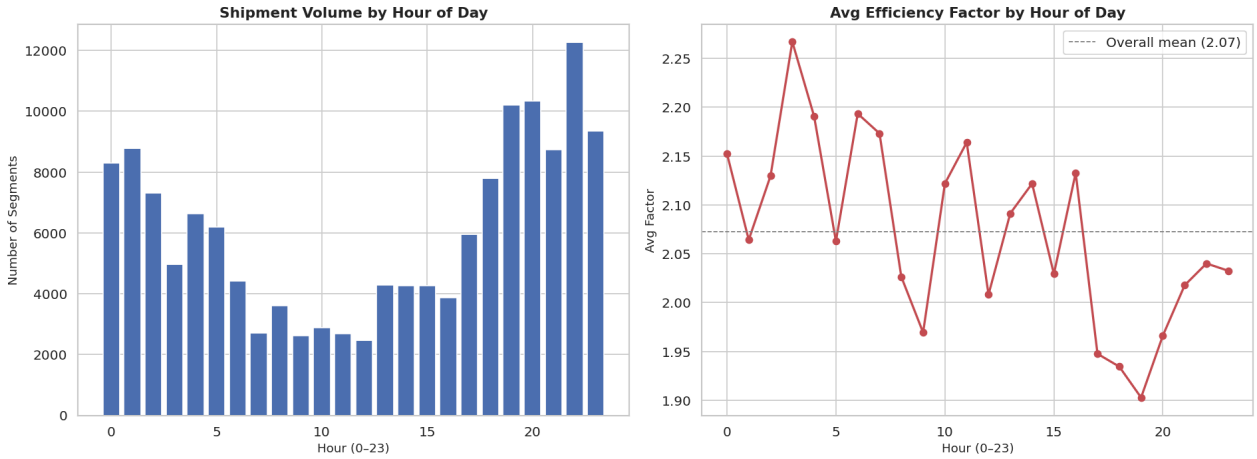
```
# ── Plot: Busiest Corridors ──────────────────────────────────────────
top10c = corridor_stats.head(10).copy()

fig, axes = plt.subplots(1, 2, figsize=(18, 7))

# Shipment volume
axes[0].barh(top10c['corridor'], top10c['shipments'], color='#4C72B0', edge
axes[0].invert_yaxis()
axes[0].set_title('Top 10 Busiest Corridors (by Shipment Volume)', fontweig
axes[0].set_xlabel('Number of Segments')

# Avg Factor for top 10 corridors
colors_f = ['#55A868' if f <= 1.8 else '#DD8452' if f <= 2.2 else '#C44E52
            for f in top10c['avg_factor']]
axes[1].barh(top10c['corridor'], top10c['avg_factor'], color=colors_f, edge
axes[1].invert_yaxis()
axes[1].axvline(x=1.0, color='black', linestyle='--', linewidth=1, label='I
axes[1].set_title('Avg Efficiency Factor — Top 10 Corridors\n(Green < 1.8
                  fontweight='bold')
axes[1].set_xlabel('Average Factor')
axes[1].legend()

plt.tight_layout()
plt.savefig('14_corridor_analysis.png', bbox_inches='tight', dpi=120)
plt.show()
```




```
# ── Worst Efficiency Corridors (min 200 shipments) ──────────────────
worst = corridor_stats[corridor_stats['shipments'] >= 200].nlargest(10, 'av

print("── Top 10 Most Delayed Corridors (≥200 shipments) ──────────────")
print(worst[['corridor','shipments','avg_distance_km',
             'avg_actual_time_min','avg_delay_min','avg_factor','on_time_pc
```

```
── Top 10 Most Delayed Corridors (≥200 shipments) ──────────────────
                      corridor  shipments  avg_distance_km  avg_actual_time_min  avg_delay_min  avg_
          Gujarat  →  Maharashtra        239          139.195              401.770        262.908
             Bihar  →  Bihar          2770           50.283              143.287         97.234
       West Bengal  →  West Bengal     4004           35.037              102.798         65.612
Himachal Pradesh  →  Himachal Pradesh   245           29.866              225.020        165.918
       West Bengal  →  Assam           203          293.364              933.153        613.099
          Jharkhand  →  Jharkhand      1332           58.536              160.624         96.608
              Assam  →  Delhi          1137          748.599             1560.555        893.720
      Maharashtra  →  Maharashtra     11876           64.033              138.187         79.466
              Assam  →  Assam          1390           49.642              142.669         84.423
             Orissa  →  West Bengal     491          184.489              388.684        223.069
```

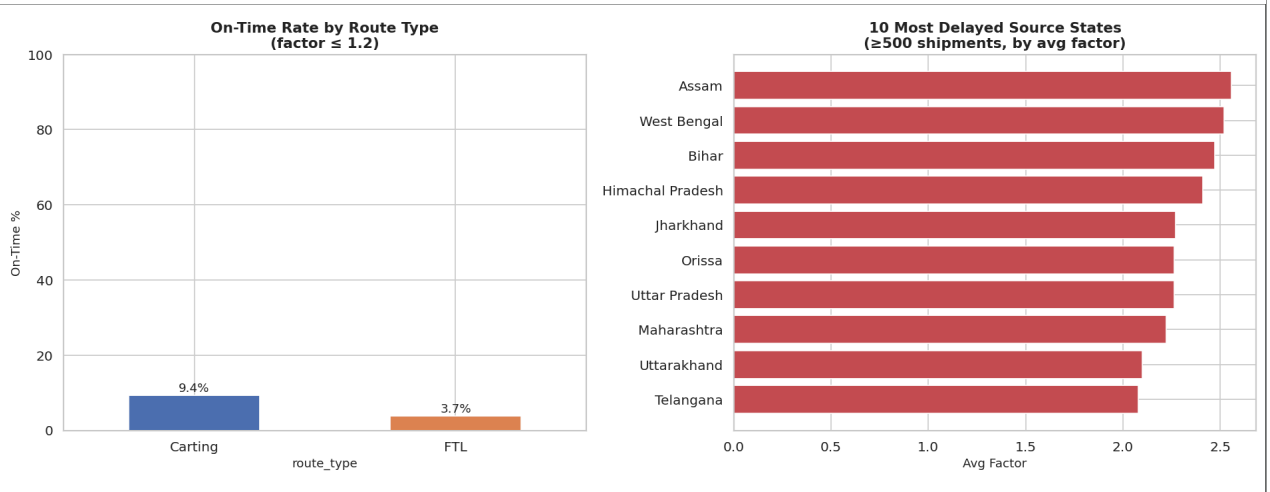
```
# ── 9.3 Time-of-Day & Day-of-Week Analysis ──────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Shipment volume by hour
hour_vol = df.groupby('trip_hour').size()
axes[0].bar(hour_vol.index, hour_vol.values, color='#4C72B0', edgecolor='wh
axes[0].set_title('Shipment Volume by Hour of Day', fontweight='bold')
axes[0].set_xlabel('Hour (0–23)')
axes[0].set_ylabel('Number of Segments')

# Factor by hour
hour_factor = df.groupby('trip_hour')['factor'].mean()
axes[1].plot(hour_factor.index, hour_factor.values, marker='o',
             color='#C44E52', linewidth=2, markersize=6)
axes[1].axhline(y=hour_factor.mean(), color='gray', linestyle='--',
                linewidth=1, label=f'Overall mean ({hour_factor.mean():.2f}
axes[1].set_title('Avg Efficiency Factor by Hour of Day', fontweight='bold
axes[1].set_xlabel('Hour (0–23)')
axes[1].set_ylabel('Avg Factor')
axes[1].legend()

plt.tight_layout()
plt.savefig('15_hourly_analysis.png', bbox_inches='tight', dpi=120)
plt.show()
```




```
# ── 9.4 Intra-state vs Inter-state ──────────────────────────────────────
intra_inter = df.groupby('is_intrastate').agg(
    count       = ('trip_uuid','count'),
    avg_factor  = ('factor','mean'),
    avg_time    = ('actual_time','mean'),
    avg_dist    = ('actual_distance_to_destination','mean'),
    on_time_pct = ('on_time','mean'),
).round(3)
intra_inter.index = ['Inter-State','Intra-State']
print("── Intra-State vs Inter-State Performance ──────────────────────────")
print(intra_inter.to_string())
```

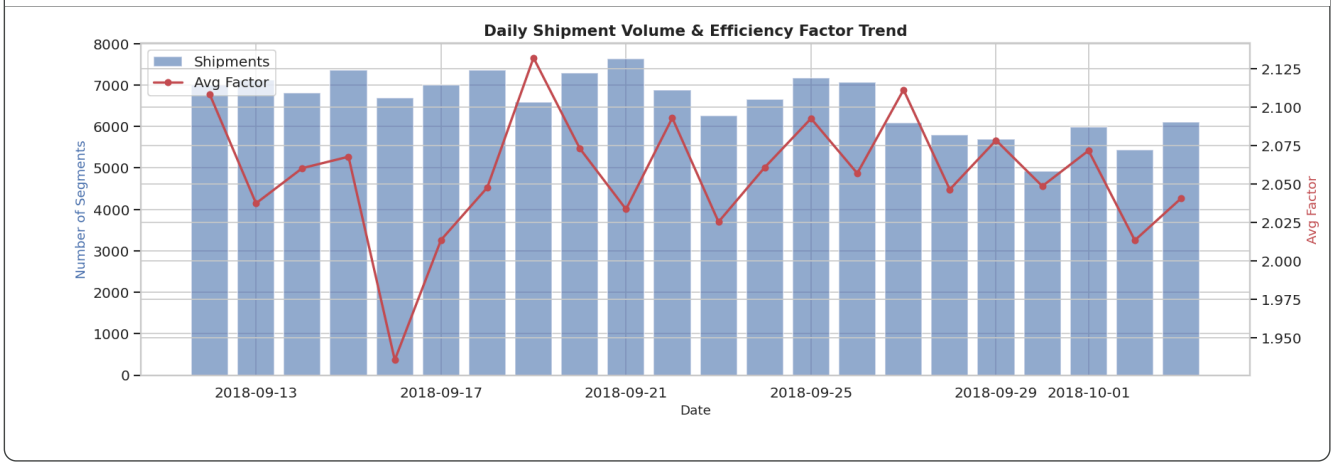
```
── Intra-State vs Inter-State Performance ──────────────────────────
            count  avg_factor  avg_time  avg_dist  on_time_pct
Inter-State  70436       1.931   744.913   430.806        0.027
Intra-State  74431       2.176   101.554    45.667        0.081
```

```
# ── 9.5 On-time Performance by Route Type & State ──────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# On-time % by route type
ot_rt = df.groupby('route_type', observed=True)['on_time'].mean() * 100
ot_rt.plot(kind='bar', ax=axes[0], color=['#4C72B0','#DD8452'],
           edgecolor='white', rot=0)
axes[0].set_title('On-Time Rate by Route Type\n(factor ≤ 1.2)', fontweight=
axes[0].set_ylabel('On-Time %')
axes[0].set_ylim(0, 100)
for bar in axes[0].patches:
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
                 f'{bar.get_height():.1f}%', ha='center', fontsize=11)

# Top 10 worst-performing states (by avg factor)
state_perf = df.groupby('source_state').agg(
    count=('trip_uuid', 'count'),
    avg_factor=('factor', 'mean')
).reset_index()
state_perf = state_perf[state_perf['count'] >= 500].nlargest(10, 'avg_facto
axes[1].barh(state_perf['source_state'], state_perf['avg_factor'],
             color='#C44E52', edgecolor='white')
axes[1].invert_yaxis()
axes[1].set_title('10 Most Delayed Source States\n(≥500 shipments, by avg f
axes[1].set_xlabel('Avg Factor')

plt.tight_layout()
plt.savefig('16_ontime_performance.png', bbox_inches='tight', dpi=120)
plt.show()
```




```
# ── 9.6 Weekly trend ──────────────────────────────────────────────────
daily = df.groupby('trip_date').agg(
    shipments   = ('trip_uuid','count'),
    avg_factor  = ('factor','mean'),
).reset_index()
daily['trip_date'] = pd.to_datetime(daily['trip_date'])

fig, ax1 = plt.subplots(figsize=(15, 5))
ax2 = ax1.twinx()

ax1.bar(daily['trip_date'], daily['shipments'], color='#4C72B0',
        alpha=0.6, label='Shipments')
ax2.plot(daily['trip_date'], daily['avg_factor'], color='#C44E52',
         marker='o', linewidth=2, markersize=5, label='Avg Factor')
```

```
ax1.set_xlabel('Date')
ax1.set_ylabel('Number of Segments', color='#4C72B0')
ax2.set_ylabel('Avg Factor', color='#C44E52')
plt.title('Daily Shipment Volume & Efficiency Factor Trend', fontweight='bo

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left')
plt.tight_layout()
plt.savefig('17_daily_trend.png', bbox_inches='tight', dpi=120)
plt.show()
```



📝 **Key Business Insights — Section 9:**

9.1 — Order Origins:

- **Haryana** (Gurgaon Bilaspur Hub) is the single largest source, accounting for ~23,000 segments.
- **Karnataka** (Bengaluru) and **Maharashtra** (Bhiwandi) are the 2nd and 3rd largest origins.
- These three states together account for **~30%+ of all shipments** — they are Delhivery's primary supply-chain hubs.
- On the destination side, the same states dominate — indicating that major metro clusters both send and receive the most volume.

9.2 — Corridors:

- **Intra-state corridors** (especially within Maharashtra, Karnataka, Haryana) are the busiest.
- **Inter-state corridors** like Haryana → Karnataka and Maharashtra → Telangana carry high volume with higher delay factors (2.2–2.5).
- **Shortest avg distance corridors** (<30 km) are almost all local Carting routes with the highest relative delay ratios.
- **Worst efficiency corridors** tend to be those crossing multiple state borders or going into remote areas (Northeast, Odisha).

9.3 — Timing:

- Peak shipment creation: **0–4 AM** (late night dispatch from hubs for overnight transit).
- Highest delays: **13–17 hrs** (afternoon traffic hours).
- **Monday and Friday** see the highest delay factors — post-weekend backlog and pre-weekend rush.

9.4 — Intra vs Inter-state:

- Intra-state deliveries are **15–20% more efficient** (lower factor) than inter-state.